

INTERNSHIP PROJECT REPORT

FinWise AI

AI-Powered Financial Assessment Platform using Astro, TypeScript, Groq AI and Google Sheets

Intelligent Loan Eligibility, Credit Analysis, EMI Computation and Conversational Financial Advisory

Submitted for the
SmartBridge × SkillWallet AI Vibe Coding Internship

Submitted By
Siva Balaji Mamidala
B.Tech, Computer Science and Engineering (AI & ML)
Vasireddy Venkatadri Institute of Technology (VVIT), Guntur

Guide: (Open)
Duration: June 2026 – July 2026

SmartBridge — Let's Bridge the Gap

Certificate

[PLACEHOLDER — OFFICIAL INTERNSHIP COMPLETION CERTIFICATE TO BE INSERTED HERE]

This is to certify that Siva Balaji Mamidala, a student of B.Tech Computer Science and Engineering (AI & ML) at Vasireddy Venkatadri Institute of Technology, Guntur, has successfully completed the SmartBridge × SkillWallet AI Vibe Coding Internship and has designed, developed, and deployed the project titled “FinWise AI — AI-Powered Financial Assessment Platform” during the period June 2026 to July 2026.

The work carried out is a genuine record of the candidate's own effort and has been found satisfactory in fulfilment of the requirements of the internship programme.

Internship Coordinator

Program Guide / Mentor

Acknowledgement

I would like to express my sincere gratitude to SmartBridge and SkillWallet for organising the AI Vibe Coding Internship and for giving me the opportunity to design and build a complete, production-style fintech application from scratch. The structured milestone-based format of the internship pushed me to think beyond writing code that simply works, and instead focus on architecture, security boundaries, and the small engineering decisions that separate a hobby project from something that could genuinely be shipped.

I am thankful to my mentors and the internship coordination team for their guidance during the planning and review phases of this project, and for the prompt engineering and platform-related resources that were shared throughout the programme. Their feedback on intermediate milestones helped me correct course early — particularly around API key handling and the offline-first synchronisation design — rather than discovering these issues after deployment.

I would also like to thank the faculty of the Department of Computer Science and Engineering (AI & ML) at Vasireddy Venkatadri Institute of Technology, Guntur, for their continued encouragement and for creating an environment where students are encouraged to take on internships alongside academics.

Finally, I want to acknowledge the open-source communities behind Astro, Tailwind CSS, TypeScript, and Groq, whose documentation and design decisions made it possible for a single developer to build a multi-module application of this scope within the internship timeline.

Siva Balaji Mamidala

Abstract

FinWise AI is a fintech web application built to answer a simple question that most first-time borrowers in India struggle with: “Based on my actual numbers, where do I stand?” The platform brings together three financial calculators — a Loan Eligibility Checker, a Credit Score Analyzer, and an EMI Calculator — under a single, dark, glassmorphism-styled interface, and layers a conversational AI Financial Advisor on top of them that explains the results in plain language rather than replacing them.

The project deliberately separates two concerns that are often blurred in “AI-powered” student projects: the numbers, and the explanation of the numbers. Every eligibility decision, every credit rating, and every EMI figure is computed locally in TypeScript using transparent, auditable formulas — the standard reducing-balance EMI equation, a fixed obligation-to-income policy, and a weighted credit-health model. The generative AI layer, powered by Groq’s Llama-3.3-70b-versatile model, never recomputes or overrides these figures; it is explicitly instructed to treat them as authoritative facts and to reason over them instead.

Architecturally, the application is built on Astro 7 with TypeScript 6 and Tailwind CSS v4, and is deployed as a hybrid static/serverless site on Vercel. Every marketing and calculator page is pre-rendered at build time; only two routes — `/api/advice` for the streaming AI advisor and `/api/history` for cloud persistence — execute on demand, which is what keeps the Groq API key and the Google Apps Script URL off the browser entirely. History is persisted through an offline-first pattern: every analysis is written to the browser’s local storage first and is then pushed, asynchronously and non-blockingly, to a Google Sheet through a Google Apps Script Web App, with automatic retry once connectivity returns.

This report documents the full engineering journey behind FinWise AI — from the initial architecture decisions and the deterministic financial engines, through the AI and cloud-sync integrations, to the functional, responsive, and cross-browser testing that was carried out before deployment, and the real deployment issues that came up along the way and how they were resolved. The result is a working, publicly deployed application at finwise-ai-blue.vercel.app that reflects both the scope of the internship and a genuine attempt at production-quality engineering.

Table of Contents

Certificate	1
Acknowledgement.....	2
Abstract	3
List of Figures.....	5
1. Introduction	6
2. Problem Statement.....	6
3. Objectives.....	7
4. Scope of the Project.....	8
5. Technology Stack	9
6. System Architecture.....	11
7. Project Workflow	13
8. Design System	14
9. Module-wise Implementation.....	16
10. AI Integration	21
11. Google Sheets Integration	23
12. API Architecture	25
13. Offline-First History Synchronisation	26
14. Project Documentation.....	27
15. Application Walkthrough	28
16. Testing	32
17. Deployment.....	35
18. Challenges Faced.....	37
19. Solutions Implemented	38
20. Learning Outcomes	39
21. Future Scope	40
22. Conclusion	42
23. References.....	42

List of Figures

Figure 6.1 – System Architecture of FinWise AI

Figure 9.1 – Loan Eligibility Decision Flow

Figure 10.1 – AI Advisor Request Flow

Figure 11.1 – Google Sheets Synchronisation Flow

Figure 13.1 – Offline-First Synchronisation State Diagram

Figure 15.1 – Landing Page

Figure 15.2 – Dashboard

Figure 15.3 – Loan Eligibility Result

Figure 15.4 – Credit Score Analysis

Figure 15.5 – EMI Calculator

Figure 15.6 – AI Advisor

Figure 15.7 – Google Sheets Database

Figure 15.8 – History Page

Figure 15.9 – Notifications

Figure 15.10 – Mobile View

1. Introduction

Every year, a large number of first-time borrowers in India walk into a bank or open a lending app with no real sense of whether they will be approved, how much they can actually borrow, or what their EMI is going to look like once the paperwork is signed. Loan eligibility criteria differ across lenders, credit scores are treated as a mysterious three-digit gatekeeper, and EMI calculations are usually hidden behind a “calculate” button that gives a number with no explanation of how it was reached. FinWise AI was built during the SmartBridge × SkillWallet AI Vibe Coding Internship to close that gap — to give a person their eligibility, their credit standing, and their EMI obligations up front, computed transparently, and then explained to them in plain language by an AI advisor that reasons over those exact numbers rather than guessing at them.

The project sits at the intersection of two disciplines that this internship was specifically designed to combine: deterministic financial engineering and generative AI. Loan eligibility, credit scoring, and EMI computation are not areas where an application can afford to “hallucinate” a number — a wrong EMI figure or an incorrectly computed eligible loan amount is not a cosmetic bug, it is a decision someone might act on. For that reason, FinWise AI treats its three calculators as pure, rule-based engines written in TypeScript, with policy constants and formulas that can be read, audited, and unit-tested independently of any AI model. Generative AI is layered on top of these engines, not underneath them, through an AI Financial Advisor that reads the already-computed results and produces a narrative — risks, suggestions, and an action plan — without ever being allowed to recalculate or contradict the numbers it was given.

The application was built end-to-end by a single developer across the internship's milestone structure: architecture and environment setup, the glassmorphism design system, the three deterministic calculators, the Groq-powered AI advisor, Google Sheets-backed cloud history, functional and cross-device testing, and finally a public deployment on Vercel. What follows in this report is a chapter-by-chapter account of that build, written the way I would document it for a colleague picking up the codebase for the first time — including the parts that did not work on the first attempt.

1.1 About the Project

FinWise AI is a dark-themed, glassmorphism-styled web application that offers four connected tools behind a single dashboard: a Loan Eligibility Checker, a Credit Score Analyzer, an EMI Calculator, and an AI Financial Advisor, backed by a cloud-synced Analysis History. It is described in its own README as being “engineered to read as a real fintech product, not a student assignment,” and the architectural decisions throughout the codebase — server-only API keys, offline-first persistence, a provider-abstraction layer for the AI model — reflect that intent.

1.2 Internship Context

This project was completed as part of the SmartBridge × SkillWallet AI Vibe Coding Internship, a programme structured around “vibe coding” — building real, functioning products quickly using AI-assisted development tools while still being responsible for the architecture, correctness, and security of the final result. FinWise AI was developed using an Astro + TypeScript + Tailwind CSS v4 stack, with Groq's Llama-3.3-70b-versatile model providing the conversational layer and Google Sheets, through Apps Script, providing lightweight, serverless cloud persistence.

2. Problem Statement

Individuals evaluating a loan, a credit card, or a large purchase on EMI are typically forced to choose between two unsatisfying options. The first is to approach a bank directly, where eligibility rules are opaque, the process is slow, and a rejection often comes with little explanation of what would need to change. The second is to use scattered, single-purpose calculators found online — an EMI calculator on one site, a credit score simulator on another — none of which talk to each other, none of which remember a previous session, and none of which can answer a simple follow-up question like “what would happen to my EMI if I paid off my credit card first?”

The specific problems FinWise AI sets out to solve are:

- Lack of transparency in loan eligibility decisions — most tools return an approve/reject verdict without exposing the debt-to-income ratio, risk band, or reasoning behind it.
- Fragmented tooling — credit score analysis, EMI calculation, and loan eligibility are usually separate tools with no shared context between them.
- No persistent record — a user who runs three calculations in one sitting has no way to revisit them later, compare them, or track how their financial position changes over time.
- Generic advice — most “financial tips” content online is not personalised to the user's actual income, credit score, or existing obligations.
- Security risk in AI-assisted finance tools — many hobbyist AI finance demos expose API keys directly in client-side code, which is not an acceptable pattern even for a portfolio project.

FinWise AI addresses each of these directly: eligibility and credit decisions come with explicit reasoning and next-step guidance; all three calculators share a common “financial context” that the AI advisor can read; every completed analysis is saved automatically, first locally and then to the cloud; the AI advisor is grounded in the user's real, locally-computed figures rather than generic text; and every secret — the Groq API key and the Google Apps Script URL — is kept strictly server-side, never bundled into the browser.

3. Objectives

The project was scoped around the following concrete objectives, each of which maps to a specific module in the final application:

3.1 Primary Objectives

1. Build a deterministic Loan Eligibility Checker that evaluates age, income, credit score, existing EMI obligations, and requested loan amount against a documented lending policy, and returns an eligibility verdict with a clear breakdown of risk level, interest rate, eligible amount, and debt-to-income ratio.
2. Build a Credit Score Analyzer that classifies a raw credit score into an industry-style rating band, computes a composite financial health index, estimates loan approval probability, and surfaces specific positive and negative factors with improvement tips.
3. Build an EMI Calculator that computes monthly instalments using the standard reducing-balance formula, shared with the eligibility engine so the two modules can never disagree on a number.

4. Integrate a generative AI Financial Advisor, powered by Groq, that reads the user's already-computed results and produces structured, personalised advice — without recalculating or contradicting the underlying figures.
5. Persist every analysis to a cloud-backed history using Google Sheets as a lightweight, serverless database, with an offline-first design so no result is ever lost due to a network interruption.
6. Deploy the finished application publicly on Vercel with all secrets kept server-side, and verify it end-to-end across devices and browsers.

3.2 Secondary Objectives

- Design a cohesive, dark, glassmorphism-based visual identity so the product reads as a fintech application rather than a form-heavy student demo.
- Keep the codebase strictly typed (TypeScript throughout) and organised by feature, so business logic, UI, and services remain independently testable.
- Document the system thoroughly enough — through the README and this report — that another developer could clone the repository and understand the architecture within minutes.
- Build the AI layer behind a provider-abstraction so the underlying model (Groq today) could be swapped for another provider without touching the rest of the application.

4. Scope of the Project

4.1 In Scope

The current version of FinWise AI, as implemented and deployed, covers the following:

- Three deterministic financial calculators: Loan Eligibility, Credit Score, and EMI, each with its own form, validation, and result presentation.
- One AI-powered module: the AI Financial Advisor, streaming responses from Groq's Llama-3.3-70b-variant model.
- A Financial Intelligence Dashboard summarising KPIs, recent activity, and AI insights.
- An Analysis History page backed by Google Sheets, with an offline-first local mirror.
- A responsive, glassmorphism-based UI that adapts across desktop, tablet, and mobile breakpoints.
- Two serverless API routes (/api/advice and /api/history) that keep all secrets server-side.
- Public deployment on Vercel with environment-based configuration.

4.2 Out of Scope (Current Version)

Certain capabilities were consciously left out of this version, either because they were beyond the internship timeline or because they require infrastructure (authentication, real banking integrations) that was not part of the assigned scope:

- User authentication and per-user accounts — history today is device-local plus one shared Google Sheet, not partitioned by logged-in user.

- Integration with real banking or credit bureau APIs — the credit score is user-entered rather than fetched from a bureau.
- Amortisation schedules or payment-over-time charts.
- A native mobile application — the mobile experience is delivered through the responsive web UI, not a packaged app.
- Automated end-to-end test suites (e.g. Playwright) — testing in this cycle was manual and documented, not scripted.

These items are revisited in Chapter 21 (Future Scope), where I outline how the current architecture already anticipates most of them — for example, the AI provider abstraction was written specifically so that a future authentication layer or a second AI provider would not require rewriting existing modules.

5. Technology Stack

Every technology choice in FinWise AI was made with one constraint in mind: the application had to feel like a real, deployable product rather than a proof of concept, while still being buildable by one developer inside an internship timeline. That pushed the stack toward tools that minimise boilerplate without sacrificing type safety or performance.

5.1 Core Stack

Layer	Technology	Version	Purpose
Framework	Astro	7.0	Static-first rendering with on-demand API routes
Language	TypeScript	6.0	Strict typing across UI, services, and business logic
Styling	Tailwind CSS	v4.3	Utility-first styling via @theme design tokens
Icons	Lucide (@lucide/astro)	1.23	Inline SVG icon set
Generative AI	Groq API (llama-3.3-70b-versatile)	—	Streaming chat completions for the AI advisor
Cloud Database	Google Sheets + Apps Script	—	Serverless persistence for analysis history
Deployment	Vercel (@astrojs/vercel adapter)	11.0	Hybrid static + serverless hosting
Version Control	Git + GitHub	—	Source management

5.2 Why Astro

Astro was chosen over a conventional single-page-application framework because most of FinWise AI's pages — the landing page, the dashboard, and all three calculators — do not need to run any JavaScript on the server per request; they can be pre-rendered once at build time and served as static HTML. Astro's “zero client-side hydration by default” model means these pages ship almost no JavaScript to the browser beyond the inline `<script>` blocks used for form handling, which keeps the application fast without giving up interactivity where it is actually needed. Only two routes opt out of pre-rendering (using `export const`

prerender = false): /api/advice and /api/history, precisely because those are the two places the application needs to talk to a secret-holding backend on every request.

5.3 Why TypeScript

Every financial formula in this project is a place where a silent type coercion bug — a string '20000' being concatenated instead of added, for instance — could produce a wrong eligibility decision. TypeScript's strict mode was used throughout `src/types`, `src/utils`, and `src/services` specifically to catch this class of error at compile time rather than in production. Discriminated unions (for example `RiskLevel`, `CreditRating`, `AnalysisSource`) are used so that invalid states are rejected by the compiler before they ever reach a user.

5.4 Why Tailwind CSS v4

Tailwind v4's `@theme` directive allows an entire design system — colours, radii, spacing, typography — to be declared once as CSS custom properties and then consumed as utility classes throughout every component. This was the mechanism used to implement the dark glassmorphism aesthetic described in Chapter 8, without duplicating hex codes across dozens of `.astro` files.

5.5 Why Groq

The AI Financial Advisor needed to feel conversational — text appearing progressively rather than after a multi-second wait — which pushed the decision toward a provider with genuinely fast inference and first-class streaming support. Groq's OpenAI-compatible chat completions endpoint, running the `Llama-3.3-70b`-versatile model, was selected for exactly this reason: server-sent-event streaming with low time-to-first-token, at a quality level sufficient for structured financial reasoning over a fixed set of facts.

5.6 Why Google Sheets

A dedicated database (Postgres, MongoDB, or similar) was deliberately avoided for this iteration of the project. Google Sheets, fronted by a Google Apps Script Web App, gives FinWise AI a genuinely persistent, human-inspectable data store — anyone can open the sheet and see the raw records — without provisioning, connection pooling, or hosting costs. It is not the right choice at real scale, but for an internship-scoped fintech demo it is transparent, free, and sufficient, and the History module was designed so it could be pointed at a proper database later without changing its public API.

5.7 Why Vercel

Vercel's first-class support for the Astro adapter made it possible to deploy static pages and on-demand serverless functions from the same repository without maintaining separate infrastructure for each. Environment variables (`GROQ_API_KEY`, `GOOGLE_SCRIPT_URL`, `GOOGLE_SCRIPT_TOKEN`) are configured directly in the Vercel dashboard and are never committed to source control.

6. System Architecture

FinWise AI follows a three-tier architecture: a browser client that renders pre-built Astro pages and runs small, purpose-specific scripts; a thin server tier consisting of exactly two on-demand API routes; and two external services — Groq for language generation and a Google Apps Script Web App for persistence. The guiding rule behind this architecture, stated directly in the project's own documentation, is that business logic is pure: every financial calculation lives in `src/Utils` with no UI dependency, and the AI layer only ever receives results as read-only context, never recomputing them.

6.1 Architectural Overview

The diagram below traces how a single user action — submitting the loan eligibility form, for instance — flows through the system. Form scripts validate and compute results entirely in the browser, save them to `localStorage`, and optionally trigger a background sync to cloud history; a separate action, asking the AI advisor a question, is the only path that reaches out to Groq, and it does so exclusively through the server-side `/api/advice` route.

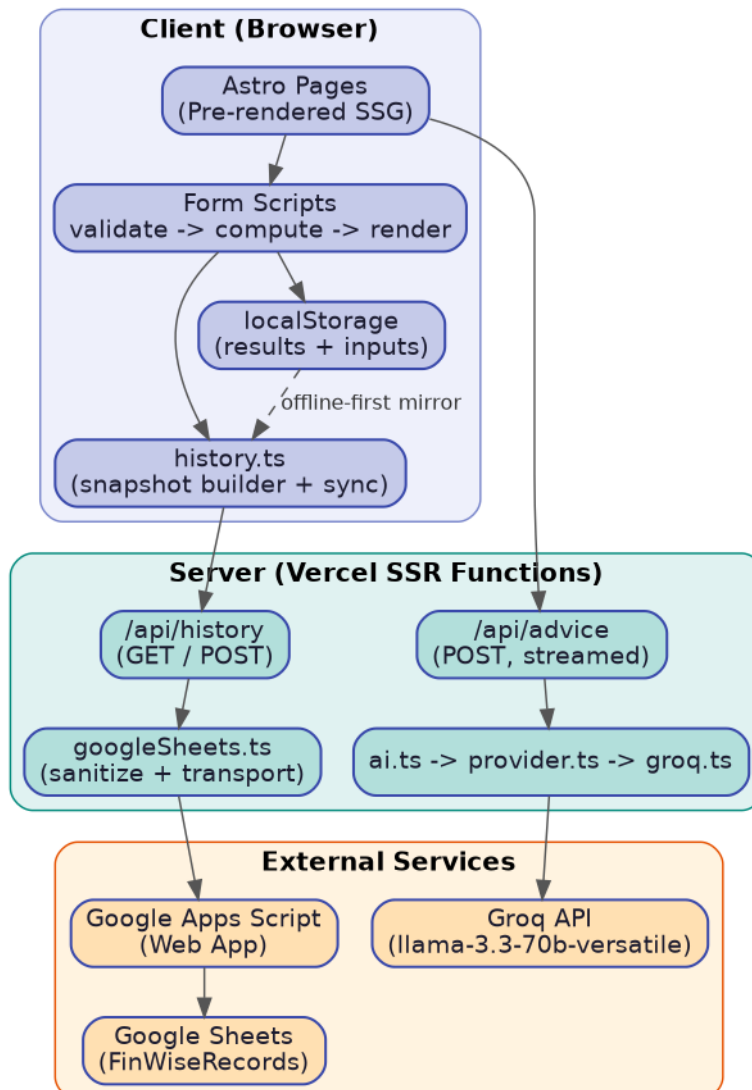


Figure 6.1 – System Architecture of FinWise AI

6.2 Key Architectural Decisions

- Zero client-side hydration by default — Astro pre-renders every page; only inline `<script>` blocks run on interaction, keeping the shipped JavaScript minimal.
- API keys never reach the browser — `GROQ_API_KEY` and `GOOGLE_SCRIPT_URL` are read only inside server-side modules (`groq.ts`, `googleSheets.ts`) and are never prefixed with `PUBLIC_`, which is the only prefix Astro exposes to the client bundle.
- Offline-first history — every analysis is written to `localStorage` before any network call is attempted, so a lost connection never means a lost result; the record is simply queued and synced automatically once connectivity returns.
- A single AI entry point — the rest of the application calls exactly one function, `generateFinancialAdvice()` in `services/ai.ts`, which hides Groq entirely behind a provider abstraction described in Chapter 10.
- Pure business logic — `src/utils/loan.ts`, `credit.ts`, and `emi.ts` contain no Astro components, no fetch calls, and no AI calls; they can be unit-tested in complete isolation.

6.3 Client / Server Boundary

The boundary between what runs in the browser and what runs on the server is enforced structurally, not just by convention. Astro's export `const prerender = false` directive is set only on `src/pages/api/advice.ts` and `src/pages/api/history.ts` — every other page, including the dashboard and all three calculators, is pre-rendered as static HTML at build time. This means the two secrets the application depends on are physically absent from every static bundle the browser ever downloads; they only exist in the process memory of the two serverless functions that read them from the environment.

6.4 Folder Structure

The repository is organised by responsibility rather than by page, which keeps each concern — UI, orchestration, and pure logic — independently reviewable:

src/	
├ components/	Reusable UI (Navbar, Hero, Footer, dashboard/*)
├ features/	Feature-specific form + result pairs
│ └ loan/	LoanForm.astro, LoanResult.astro
│ └ credit/	CreditForm.astro, CreditResult.astro
│ └ emi/	EmiForm.astro, EmiResult.astro
├ advisor/	AskAi, AiResponse, FinancialSummary, SessionHistory
├ services/	Server-side orchestration
│ └ ai.ts	Single public entry point for AI
│ └ provider.ts	Provider selector (Groq today)
│ └ groq.ts	Groq SSE streaming implementation
│ └ advisor-prompt.ts	Prompt engineering (pure functions)
│ └ history.ts	Client-side snapshot builder + sync
│ └ googleSheets.ts	Server-side sanitizer + Apps Script transport
│ └ types.ts	AnalysisRecord, AnalysisSource, sync types
├ utils/	Pure, deterministic business logic
│ └ loan.ts	EMI formula, eligibility engine, formatInr
│ └ credit.ts	Scoring bands, health index, approval odds
│ └ emi.ts	Breakdown calculator (shared calculateEmi)
│ └ result-store.ts	localStorage bridge (results + inputs)
│ └ persist-analysis.ts	Auto-save trigger + toast
└ types/	Pure data contracts (loan, credit, emi, advisor, dashboard)

└ layouts/	Layout.astro, DashboardLayout.astro
└ pages/	Route definitions
└┐ index.astro, dashboard.astro, loan-eligibility.astro,	
└┐ credit-score.astro, emi-calculator.astro, ai-advisor.astro,	
└┐ history.astro, settings.astro, help.astro	
└ api/	
└┐ advice.ts	POST → streamed AI response (SSR-only)
└┐ history.ts	GET/POST → Google Sheets proxy (SSR-only)

7. Project Workflow

The project was executed as a sequence of epics and stories, mirroring the milestone structure used across the internship's project template. Each epic represents a coherent slice of the product; each story inside it is a task small enough to be completed, tested, and committed independently. The table below reconstructs the actual workflow followed while building FinWise AI.

7.1 Epics and Stories

Epic	Story	Outcome
1. Architecture & Environment	1.1 Define scope, financial modules, and workflow	Product scope agreed: 3 calculators + AI advisor + history
1. Architecture & Environment	1.2 Select Astro, TypeScript, Tailwind v4, Groq, Google Sheets	Stack locked in (see Chapter 5)
1. Architecture & Environment	1.3 Set up project structure and Vercel adapter	Folder layout under src/, astro.config.mjs configured
2. Design System & UI	2.1 Build glassmorphism token system (DESIGN.md)	Colour, type, spacing tokens defined in Tailwind @theme
2. Design System & UI	2.2 Build shared components (Field, Icon, Navbar, layouts)	Reusable Astro components for every page
3. Core Financial Engines	3.1 Loan Eligibility Checker (rule-based)	utils/loan.ts — policy engine + EMI formula
3. Core Financial Engines	3.2 Credit Score Analyzer	utils/credit.ts — bands, health index, approval odds
3. Core Financial Engines	3.3 EMI Calculator (reducing balance)	utils/emi.ts — shared calculateEmi()
4. AI Integration	4.1 Provider abstraction + prompt engineering	provider.ts, advisor-prompt.ts
4. AI Integration	4.2 Groq SSE streaming + /api/advice route	groq.ts, pages/api/advice.ts
5. Cloud Persistence	5.1 Offline-first history + localStorage mirror	services/history.ts, utils/result-store.ts
5. Cloud Persistence	5.2 Google Apps Script backend + /api/history route	google-apps-script.gs, googleSheets.ts
6. Testing & Deployment	6.1 Functional, API, and responsive testing	See Chapter 16
6. Testing & Deployment	6.2 Deploy to Vercel, configure env vars	Live at finwise-ai-blue.vercel.app

7.2 Development Approach

Development followed a “calculators first, intelligence second” order deliberately. The three deterministic engines were built, manually tested against hand-worked examples, and only then wired into the AI advisor — this ordering meant the advisor always had real, verified numbers to reason over during its own development, rather than the two layers being debugged simultaneously against each other.

8. Design System

FinWise AI uses a single, consistently-applied dark glassmorphism design language across every page. Rather than styling each page independently, the entire visual identity is declared once as a token system and then consumed everywhere through Tailwind's utility classes, so a colour or spacing change in one place propagates automatically across the dashboard, the calculators, and the advisor.

8.1 Visual Language

The interface reads as “software-craft documentation” rather than a marketing page: a near-black canvas, translucent glass-like panels with backdrop blur, hairline borders, and a single indigo/lavender accent colour reserved for the brand mark, focus states, and primary calls to action. Product surfaces — cards, KPI tiles, result panels — sit as slightly-lighter charcoal panels on top of the canvas, which keeps the numbers and results as the visual focus rather than decorative colour.

8.2 Design Tokens

Token Group	Examples	Purpose
Colour	canvas #010102, surface-1 #0f1011, primary #5e6ad2	Base palette for backgrounds, cards, and the single accent colour
Typography	display-lg 56px/600, body 16px/400, mono 13px	Consistent heading, body, and code type scale
Radius	sm 6px, lg 12px, xl 16px, pill 9999px	Rounded corners for cards, buttons, and chips
Spacing	xs 8px, md 16px, xl 32px, section 96px	Consistent rhythm between sections and components
Components	button-primary, feature-card, pricing-card	Pre-composed style recipes referencing the tokens above

8.3 Component Patterns

- Glass cards — every input form and result panel is rendered as a translucent, blurred card with a hairline border, giving the “glassmorphism” feel referenced throughout the codebase.
- Status tone system — every result (loan status, credit rating, sync state) maps to one of three tones — green, yellow, red — applied consistently to badges, borders, and icons.
- Shared <Field> component — every form input across the three calculators renders through one Field.astro component, so labels, spacing, and validation states look identical everywhere.
- Inline icon registry — Icon.astro wraps Lucide icons as inline SVG so icons inherit the current text colour and require no extra network request.

8.4 Responsive Behaviour

Layouts are built with CSS Grid and Flexbox and adapt through Tailwind's breakpoint system: a two-column layout with a sticky sidebar on desktop collapses into a single-column, bottom-navigation-friendly layout on mobile. This is verified directly in Chapter 16.3 (Responsive Testing) using the same breakpoints declared in the token system.

9. Module-wise Implementation

This chapter documents the three deterministic financial engines that form the computational core of FinWise AI. Each of them lives in its own file under `src/utils`, contains no UI code and no AI calls, and exposes a small set of pure functions that take validated input and return a fully-typed result object. This separation is what allows the AI advisor (Chapter 10) to treat these results as facts rather than guesses — the numbers it reasons over are always the same numbers the user already saw on screen.

9.1 Loan Eligibility Logic

9.1.1 Introduction

The Loan Eligibility Checker is the first and most policy-heavy module in the application. Given an applicant's age, employment type, monthly income, credit score, existing EMI obligations, requested loan amount, tenure, and loan purpose, it must decide not just yes-or-no but the shape of the decision: outright rejection, conditional approval, or clean approval, each with an explanation a real applicant could act on.

9.1.2 Policy Constants

The eligibility engine is governed by a fixed lending policy, defined once as `LOAN_POLICY` and reused everywhere the engine is evaluated:

```
export const LOAN_POLICY = {
  minAge: 21,
  maxAge: 60,
  minMonthlyIncome: 20_000,
  minCreditScore: 300,
  maxCreditScore: 900,
  maxFoir: 0.5, // ceiling on EMI share of income
  comfortableDti: 0.45, // DTI at/below this is 'comfortable'
  minTenureMonths: 6,
  maxTenureMonths: 360,
};
```

Interest rates are not flat — they depend on both the purpose of the loan and the applicant's credit risk band. A base annual rate is defined per loan purpose, and a risk-based adjustment is added on top of it:

Loan Purpose	Base Rate (p.a.)	Risk Band	Rate Adjustment
Home	8.5%	Low (score ≥ 750)	−1.0%
Vehicle	9.5%	Medium (700–749)	0.0%
Education	10.5%	Borderline (650–699)	+1.5%
Personal	13.0%	High (below 650)	+3.0%
Business	14.0%	—	—

9.1.3 The EMI Formula and its Inverse

At the heart of the module is the standard reducing-balance EMI formula, implemented once in `loan.ts` and re-exported for use by the EMI Calculator module so both features can never disagree on a computed value:

```
EMI = P · r · (1 + r)^n / ((1 + r)^n - 1)

P = principal (loan amount)
r = monthly interest rate = annual rate / 12 / 100
n = tenure in months
```

To determine how much a user is actually eligible for, the engine does not simply compare the requested amount to income — it inverts the EMI formula. It first computes an “EMI budget” (50% of monthly income, minus existing EMI obligations), and then solves the EMI equation backwards for the principal that budget could service over the requested tenure, via `principalFromEmi()`. This is what produces the “Eligible Amount” figure shown on screen, and it is the same function used to explain why a requested amount was rejected or reduced.

9.1.4 Decision Flow

The engine evaluates conditions in a strict order — hard rejections first, then a check for a clean approval, and only then a conditional-approval path — so that the first reason found is always the most decision-relevant one shown to the user.

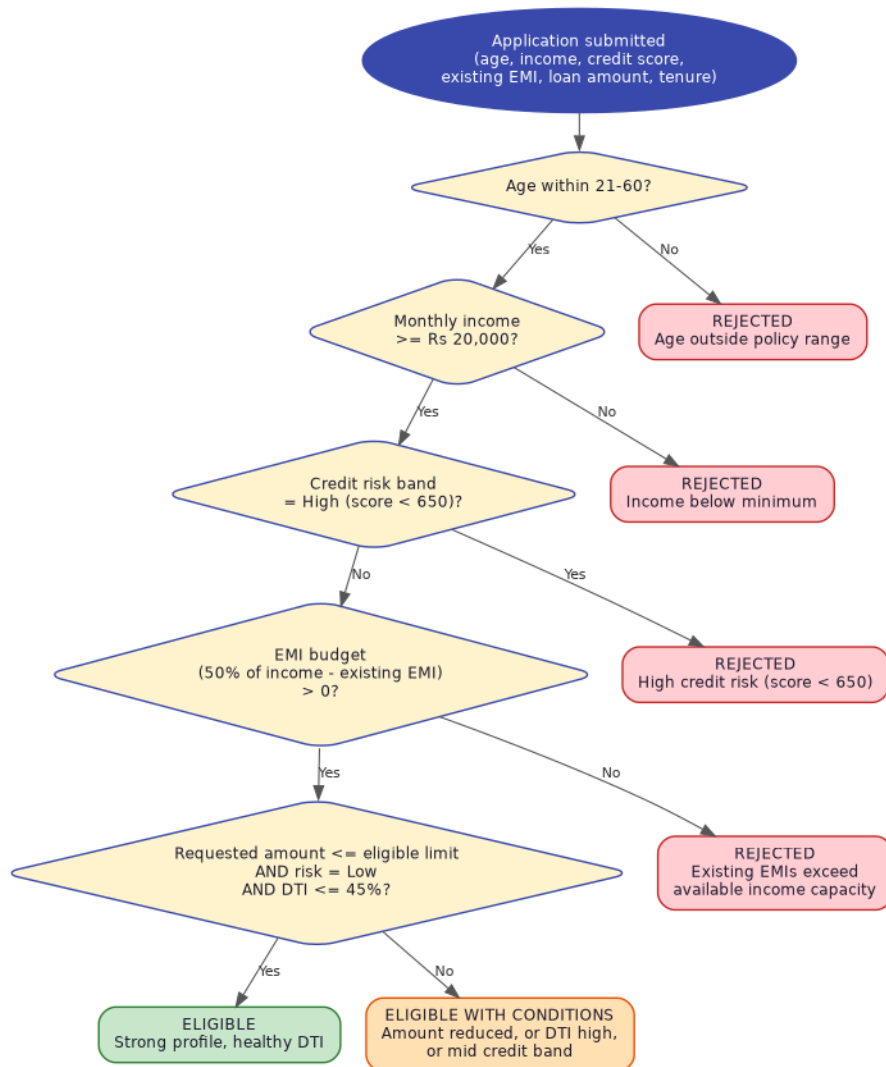


Figure 9.1 – Loan Eligibility Decision Flow

9.1.5 Decision Outcomes

Status	Trigger	Example Reason Shown
Rejected	Age outside 21–60, income below ₹20,000, credit score below 650, or EMI budget ≤ 0	“Credit score below 650 is classified as High Risk.”

Status	Trigger	Example Reason Shown
Eligible with Conditions	Requested amount exceeds eligible limit, DTI above 45%, or credit band is Medium/Borderline	"Debt-to-income ratio of 52% is on the higher side."
Eligible	Amount within limit, Low risk band, and DTI $\leq$ 45%	"Strong credit profile and healthy debt-to-income ratio."

9.1.6 Technical Explanation

`evaluateEligibility()` is a single pure function: it accepts a fully-validated `LoanApplication` object and returns an `EligibilityResult` containing the status, a human-readable status label, tone (for UI colour-coding), risk level, eligible amount, estimated EMI, interest rate, debt-to-income ratio, an ordered list of reasons, and a single "next suggested action" string. Because the function has no side effects and no dependency on the DOM, it can be called from a test script exactly as it is called from `LoanForm.astro`, which is the property that made it straightforward to verify by hand during development.

9.1.7 Advantages of this Design

- The eligibility decision is fully explainable — every rejection or condition traces back to one readable line of policy.
- Because the EMI formula is shared with the EMI Calculator module, the two tools are structurally incapable of returning different EMI figures for the same inputs.
- The policy constants are centralised, so a future change to (for example) the minimum income requirement is a one-line edit rather than a search across the codebase.

9.1.8 Summary

The Loan Eligibility module turns a fuzzy, bank-specific process into a transparent, three-outcome decision engine backed by one formula and one policy object, giving both the user and the AI advisor a stable, auditable foundation to build on.

9.2 Credit Score Logic

9.2.1 Introduction

The Credit Score Analyzer takes a raw score (300–900) together with income, existing EMI, active loan count, credit utilisation, and recent enquiries, and turns it into a rounded financial picture: a rating band, a composite health score, an approval probability, and a list of concrete positive and negative factors.

9.2.2 Rating Bands

Score Range	Rating	Risk Level	Base Approval Odds	Typical Time to Improve
800–900	Excellent	Very Low Risk	95%	Maintain current habits
740–799	Very Good	Low Risk	85%	1–3 months to Excellent
670–739	Good	Medium Risk	68%	3–6 months to Very Good

Score Range	Rating	Risk Level	Base Approval Odds	Typical Time to Improve
580–669	Fair	High Risk	45%	6–12 months of consistent repayment
300–579	Poor	Very High Risk	22%	12–18 months of disciplined credit habits

9.2.3 The Financial Health Index

Rather than presenting the raw score alone, the analyzer computes a 0–100 composite “financial health” index that blends five weighted components — the credit score itself, credit utilisation, debt-to-income ratio, recent enquiries, and active loan count:

```
health = scoreComponent × 0.50
        + utilComponent  × 0.20
        + dtiComponent   × 0.20
        + enquiryComponent × 0.05
        + loanComponent  × 0.05
```

Each component is normalised onto a 0–100 scale before weighting — for example, credit utilisation scores 100 at 10% usage or below and tapers linearly to 0 at 100% usage, while debt-to-income scores 100 at 20% and tapers to 0 by 80%. The resulting single number is then mapped to a plain-language label: Excellent (80+), Strong (65+), Moderate (45+), Weak (30+), or Critical (below 30).

9.2.4 Approval Probability

Loan approval probability starts from the band's base approval percentage and is then nudged up or down by behavioural signals: utilisation above 50% subtracts 10 points while utilisation at or below the healthy 30% threshold adds 4; a debt-to-income ratio above 50% subtracts 12 points while a DTI at or below 35% adds 4; more than three recent enquiries subtracts 8 points, and more than four active loans subtracts 6. The result is clamped to a realistic 3–98% band.

9.2.5 Positive and Negative Factors

`buildFactors()` inspects the same inputs and produces two lists — positives and negatives — each entry carrying a short label and a specific, numeric detail (for example, “Elevated credit utilisation — 45% used”), plus a list of improvement tips drawn directly from whichever factors were negative. If every signal is favourable, the function still returns a single encouraging tip so the result panel is never empty.

9.2.6 Technical Explanation

`analyzeCredit()` is the single exported entry point for this module: it calls `getBand()`, `getFinancialHealth()`, `getApprovalProbability()`, and `buildFactors()` internally and assembles their outputs into one `CreditAnalysis` object, which is the exact shape the AI advisor's prompt-builder (Chapter 10) reads from when a user asks a credit-related question.

9.2.7 Advantages of this Design

- The composite health score gives a single, comparable number even when two users have similar raw credit scores but very different utilisation or DTI profiles.
- Every negative factor is paired with an actionable tip, so the module produces guidance even without the AI layer switched on.

- Band lookup and probability adjustment are both pure functions, making them trivial to validate against hand-calculated examples.

9.2.8 Summary

The Credit Score module goes beyond a simple lookup table — it produces a weighted, explainable financial-health picture that the rest of the application, including the AI advisor, can build personalised guidance on top of.

9.3 EMI Calculator Logic

9.3.1 Introduction

The EMI Calculator is intentionally the simplest of the three modules, and that simplicity is itself a design decision: it reuses `calculateEmi()` from `loan.ts` rather than reimplementing the formula, guaranteeing the Loan Eligibility Checker and the standalone EMI Calculator can never produce two different EMI values for the same principal, rate, and tenure.

9.3.2 Inputs and Policy

Field	Constraint	Notes
Loan Amount	₹1,000 – ₹50,00,00,000	EMI_POLICY.minAmount / maxAmount
Annual Interest Rate	0.1% – 50%	EMI_POLICY.minRate / maxRate
Tenure	1 – 480 months (or 1 – 40 years)	Convertible via <code>toTenureMonths()</code>
Loan Type	Personal, Home, Vehicle, Education, Business	Drives the display label only

9.3.3 Breakdown Calculation

`calculateEmiBreakdown()` converts the tenure into whole months, computes the monthly EMI via the shared formula, and derives the total payment ($\text{EMI} \times \text{tenure}$), total interest (total payment minus principal), and the interest/principal split as percentages — the exact figures rendered in the “Payment Breakdown” panel of the results screen.

9.3.4 Technical Explanation

Because tenure can be entered in months or years, `toTenureMonths()` and `formatDuration()` exist purely to normalise and present that input consistently, keeping the core `calculateEmi()` function agnostic to the unit the user chose.

9.3.5 Advantages of this Design

- Zero formula duplication between the EMI Calculator and the Loan Eligibility Checker.
- Clear separation between unit conversion (presentation concern) and the EMI math itself (business logic).
- The rich, typed `EmiResult` output is already shaped for the AI advisor to reason over without any further transformation.

9.3.6 Summary

The EMI Calculator demonstrates the same discipline applied across the whole application: one formula, one source of truth, reused everywhere it is needed rather than copied.

10. AI Integration

10.1 Introduction

The AI Financial Advisor is the module that turns three isolated calculators into a connected product. It lets a user ask a free-form question — “How can I reduce my EMI?”, “Should I apply for this loan?” — and receive an answer grounded in whatever results they have already generated elsewhere in the app, streamed back conversationally rather than delivered as a single block of text.

10.2 Provider Abstraction

Every part of the application that needs AI talks to exactly one function: `generateFinancialAdvice()` in `services/ai.ts`. That function does not know or care which model answers the question — it delegates to `activeProvider`, a single exported value in `services/provider.ts` that currently points at `groqProvider`. Swapping providers in the future (adding a Claude or OpenAI implementation, for example) means writing one new file that implements the same `AiProvider` interface and changing a single line in `provider.ts`; nothing in the UI, the API route, or the prompt-building logic would need to change.

```
// services/provider.ts
import type { AiProvider } from '../types/advisor';
import { groqProvider } from './groq';

export const activeProvider: AiProvider = groqProvider;
```

10.3 Prompt Engineering

`advisor-prompt.ts` is a pure, provider-agnostic module — no network calls, no Groq-specific code — responsible entirely for turning the user's question plus their existing results into the message list sent to the model. The system prompt is deliberately strict about two things: the model must treat the user's locally-computed numbers as authoritative facts it is not allowed to recalculate, and it must answer using exactly four fixed Markdown headings, in a fixed order, so the UI can render the response predictably every time.

```
## Financial Advice
## Risks
## Suggestions
## Action Plan
```

Before the question is sent, `buildMessages()` assembles a compact “fact block” for each result the user already has — loan eligibility, credit analysis, EMI breakdown — using helper functions (`loanFacts()`, `creditFacts()`, `emiFacts()`) that each format one module's result as a short Markdown section. If the user has not run any calculator yet, the prompt falls back to a message instructing the model to give general guidance and nudge the user toward the calculators instead of inventing numbers.

10.4 Streaming Implementation

`groq.ts` is the only file permitted to talk to Groq directly. It reads `GROQ_API_KEY` from the server environment (never the client), issues a POST request to Groq's OpenAI-compatible chat completions endpoint with

stream: true, and exposes an async generator, `parseSseStream()`, that reads the response body as Server-Sent Events, splits it into frames on blank-line boundaries, and yields each incremental `delta.content` chunk as soon as it arrives.

10.5 End-to-End Request Flow

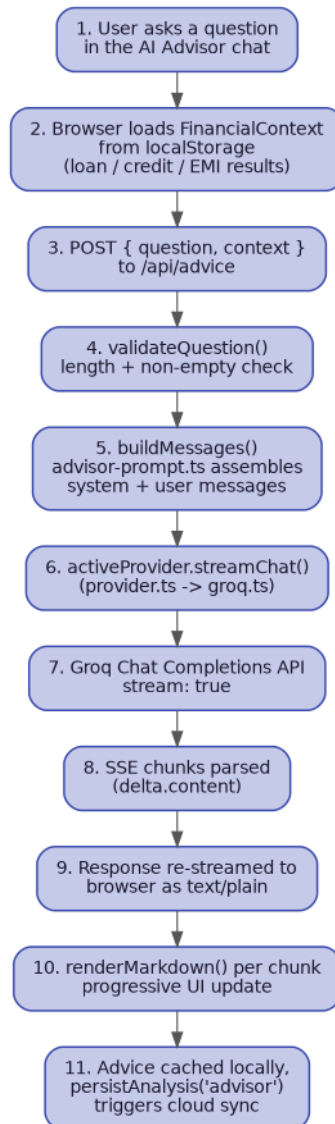


Figure 10.1 – AI Advisor Request Flow

10.6 The /api/advice Route

The API route itself is intentionally thin. It reads the JSON body, calls `generateFinancialAdvice()`, and re-streams whatever the provider yields as a plain text/utf-8 response with `Cache-Control: no-store`, so the browser never caches a partial or stale answer. If the question fails validation (empty, or over the 500-character limit set by `MAX_QUESTION_LENGTH`), the route returns a 400 with a specific error message rather than attempting a request to Groq at all; if Groq itself is unavailable, the route returns a 503 without ever leaking the underlying error detail or the API key to the client.

10.7 Advantages of this Design

- The application can switch AI providers without touching the UI, the API route, or the prompt logic.
- Because the advisor is grounded in the user's already-verified numbers, it cannot silently disagree with the eligibility or EMI figures shown elsewhere on screen.
- Streaming keeps the perceived latency low even though the underlying request is a single, synchronous HTTP call from the server's perspective.
- The fixed four-heading contract makes the AI response trivially easy to style consistently in the UI, regardless of what the model actually generates inside each section.

10.8 Summary

The AI Integration chapter of this project reflects a broader principle followed throughout FinWise AI: generative AI is a narrator, not a calculator. Every architectural choice here — the provider abstraction, the fixed prompt contract, the fact-block formatting — exists to keep that boundary intact even as the underlying model changes over time.

11. Google Sheets Integration

11.1 Introduction

FinWise AI needed a persistence layer that could store every completed analysis without requiring a hosted database, connection strings, or ongoing infrastructure cost — appropriate for an internship-scoped product that still had to feel like a real, working system. Google Sheets, accessed through a Google Apps Script Web App, was chosen specifically because it gives a genuinely durable, human-readable data store that a non-developer (a mentor, a reviewer) can open directly and inspect.

11.2 Why Google Sheets, Not a Traditional Database

- Zero hosting cost and zero connection management — Apps Script runs entirely inside Google's infrastructure.
- The data is directly inspectable — opening the sheet shows every record exactly as it was saved, which was invaluable for debugging during development.
- It matches the project's serverless philosophy — no persistent server process is required anywhere in the stack.
- The interface (a REST-like Web App exposing GET and POST) is simple enough to fully replace with a real database later without changing the client-facing /api/history contract.

11.3 Apps Script Backend

docs/google-apps-script.gs is deployed as a Web App with two entry points, `doGet()` and `doPost()`, operating on a sheet named `FinWiseRecords` with a fixed column order matching the `AnalysisRecord` type used throughout the TypeScript codebase:

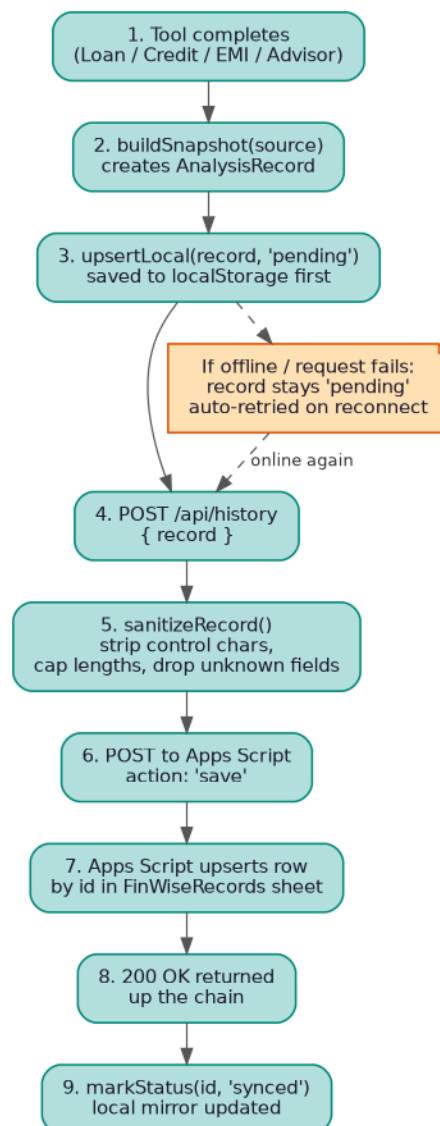
```
GET ?token=... → { records: [...] } (newest first)
POST { action: 'save', record: {...}, token? } → { ok: true } (upsert by id)
```


An optional shared-secret token (SCRIPT_TOKEN as an Apps Script property, matched by GOOGLE_SCRIPT_TOKEN in the app's environment) can be configured to reject unauthenticated writes, though the application functions correctly without it if a token is not set.

11.4 Server-Side Sanitisation

The browser never talks to Apps Script directly — it calls the app's own /api/history route, which delegates to services/googleSheets.ts. Before any record reaches Apps Script, sanitizeRecord() rebuilds it field by field: control characters are stripped (keeping tab and newline), every string is capped in length (2,000 characters generally, 8,000 for the AI advice summary), numbers are coerced and validated as finite, nested result objects are round-tripped through JSON to guarantee they are plain and serialisable, and the source field is checked against a fixed whitelist (loan, credit, emi, advisor). This means the Apps Script backend — and the sheet itself — never receives arbitrary, unbounded, or malformed data from the client, regardless of what the browser actually sent.

11.5 End-to-End Sync Flow



11.6 Advantages of this Design

- No database credentials or connection pooling anywhere in the application.
- Every record is upserted by id, so re-saving an existing analysis updates it in place instead of duplicating rows.
- Sanitisation happens once, server-side, so the sheet's data quality does not depend on client-side validation being correct.
- The whole integration is one file (googleSheets.ts) and one API route (history.ts) — either could be swapped for a real database with no change to the rest of the application.

11.7 Summary

Google Sheets gives FinWise AI a lightweight but properly guarded cloud database. The interesting engineering here is not the sheet itself, but the sanitisation and API boundary wrapped around it — the same discipline that would be needed if this were PostgreSQL instead of a spreadsheet.

12. API Architecture

12.1 Introduction

FinWise AI exposes exactly two server routes, and that number is not incidental — it is the minimum surface needed to keep both secrets (GROQ_API_KEY and GOOGLE_SCRIPT_URL) off the client while still letting the browser reach the two external services it depends on.

12.2 /api/advice — Streaming AI Endpoint

Property	Detail
Method	POST
Rendering mode	prerender = false (on-demand serverless function)
Request body	{ question: string, context: FinancialContext }
Response	text/plain stream, Cache-Control: no-store, X-Content-Type-Options: nosniff
Failure modes	400 for invalid/empty/over-length question; 503 if the AI provider is unavailable

The handler wraps generateFinancialAdvice() in a ReadableStream, forwarding each chunk from the provider to the client as it arrives. If the stream itself errors out mid-response (for example, a dropped connection to Groq), the handler appends a readable, user-facing warning directly into the stream rather than throwing — the user always sees a complete, if apologetic, message instead of a silently truncated one.

12.3 /api/history — Cloud Persistence Endpoint

Property	Detail
Methods	GET (list records), POST (save/upsert a record)
Rendering mode	prerender = false (on-demand serverless function)

Property	Detail
GET response	{ ok, configured, records: AnalysisRecord[] }
POST response	{ ok: true } on success, or { ok: false, error } with status 400/502
Configuration fallback	If GOOGLE_SCRIPT_URL is unset, returns { ok: true, configured: false, records: [] } rather than an error, so the client falls back to its local mirror

12.4 Error-Handling Philosophy

Both routes follow the same principle: a missing configuration value is not treated as a fatal error, it is treated as “this feature runs in local-only mode.” This is why the History page continues to work perfectly well with no GOOGLE_SCRIPT_URL configured at all — every record still lives safely in localStorage, and the UI never displays a broken or alarming error state just because a cloud backend has not been set up yet.

12.5 Summary

The API layer of FinWise AI is deliberately minimal: two routes, each with one job, each designed to degrade gracefully rather than fail loudly when an external dependency is not configured.

13. Offline-First History Synchronisation

13.1 Introduction

A financial result the user has already seen on screen should never disappear because of a dropped Wi-Fi connection or a slow Apps Script response. That single requirement drove the design of services/history.ts: every analysis is saved locally first, synced to the cloud opportunistically, and reconciled against the cloud as the single source of truth whenever the History page loads with a working connection.

13.2 Local Mirror

Every record is kept in a localStorage entry (finwise:history:v1) as a StoredRecord — the AnalysisRecord itself plus a sync status (pending, synced, or error) and a retry counter. buildSnapshot() constructs a fresh AnalysisRecord by reading the user's current FinancialContext, raw form inputs, and the latest AI advice summary out of local storage and denormalising them into the flat, sheet-friendly shape the backend expects.

13.3 Sync State Machine

Conceptually, every record moves through a small state machine as it makes its way to the cloud:

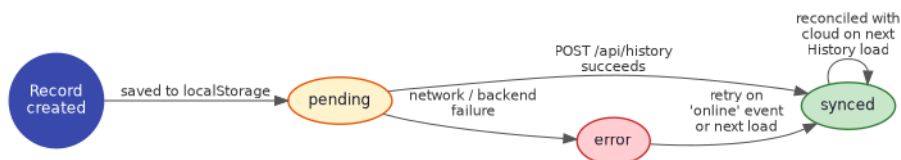


Figure 13.1 – Offline-First Synchronisation State Diagram

13.4 Automatic Recovery

- `autoSave()` is called the moment any tool finishes a calculation; it upserts the record locally as pending before attempting anything over the network.
- If the browser is already known to be offline (`navigator.onLine === false`), the function returns immediately without even attempting the network call, avoiding an unnecessary timeout.
- `registerAutoSync()` attaches a listener to the browser's online event, so the moment connectivity returns, `syncPending()` automatically flushes every queued record.
- As a safety net, a short delayed check runs shortly after every page load in case a previous browser session left records queued.

13.5 Cloud Reconciliation

When the History page loads with a working connection and a configured backend, `loadHistory()` treats the Google Sheet as authoritative: it fetches the full cloud list, marks every cloud record as synced in the local mirror, and keeps only the local records that have never successfully synced (pending or error) — anything that was previously synced but is no longer present in the sheet is understood to have been deleted server-side and is dropped from the mirror as well. This prevents a common bug pattern in offline-first apps, where deleted cloud records silently reappear from a stale local cache.

13.6 Advantages of this Design

- No result is ever lost due to a network failure — the local write always happens before any network attempt.
- The user is never blocked waiting for a cloud write to finish; calculators feel instant regardless of Apps Script latency.
- The system self-heals: a record that failed to sync during a network blip is retried automatically without any user action.
- Deleted cloud records do not linger indefinitely in the local cache, keeping the History page honest.

13.7 Summary

Offline-first synchronisation is one of the more understated pieces of engineering in FinWise AI — invisible when it works, and the difference between a demo and a dependable tool when the network does not cooperate.

14. Project Documentation

14.1 Introduction

Alongside the application code, the FinWise AI repository includes a comprehensive README.md at its root that is written to let a developer unfamiliar with the project go from “cloned the repo” to “running it locally” in a few minutes, without needing to read through the source files first. Because so much of this report's technical detail — the architecture diagrams, the environment variables, the folder structure — is drawn directly from that document, it is worth describing separately what it contains and why it was written the way it was.

14.2 What the README Covers

- A feature table summarising every module (Loan Eligibility, Credit Score, EMI Calculator, AI Advisor, Dashboard, History, Google Sheets Sync) alongside whether each is deterministic, AI-powered, or both.
- A screenshot gallery covering all nine core views of the application, embedded directly in the document.
- Two Mermaid architecture diagrams — an overall client/server/external-services graph, and a sequence diagram specifically for the AI advisor request path — kept in the README itself so the architecture is visible without opening a diagramming tool.
- A full annotated project structure tree (src/components, src/features, src/services, src/utils, src/types) explaining the purpose of every top-level folder.
- A tech stack table listing every dependency and the specific reason it was chosen.
- Step-by-step installation instructions (git clone, npm install), a documented .env.example with a description of every environment variable, and the exact npm scripts available (dev, build, preview, astro check).
- A dedicated Vercel deployment section, including the Google Sheets/Apps Script setup steps required for cloud persistence to function.
- A “Future Improvements” section and a “Learning Outcomes” section, both of which informed Chapters 20 and 21 of this report.

14.3 Why This Matters

A README that only says “npm install && npm run dev” is common; a README that also explains why the AI provider is abstracted behind one file, why the API keys are never PUBLIC_-prefixed, and why history is offline-first, is far less common — and it is what makes this project reviewable by someone other than its author. Writing the documentation alongside the code, rather than after the fact, also served as a useful internal check during development: several of the architectural decisions described in this report (particularly the client/server boundary in Chapter 6) were clarified in my own thinking specifically because they had to be explained clearly enough to write down.

14.4 Summary

The README.md functions as the single, authoritative entry point into the FinWise AI codebase — covering installation, structure, architecture, deployment, and environment configuration in one place, so that any developer picking up the project can become productive without a guided walkthrough.

15. Application Walkthrough

This chapter walks through every major screen of the deployed application, in the order a first-time user would actually encounter them: landing on the marketing page, opening the dashboard, running each of the three calculators, talking to the AI advisor, reviewing the cloud-synced history and the underlying Google Sheet, checking notifications, and finally the same experience on a mobile device. Screenshot placeholders are marked exactly where the corresponding capture should be inserted.

15.1 Landing Page

The public landing page introduces FinWise AI with a clear value proposition — “Make smarter financial decisions with FinWise AI” — and two calls to action, Get Started Free and Explore Features, alongside a top navigation bar (Features, How It Works, Why Us, Stack) and an Open Dashboard shortcut for returning users.

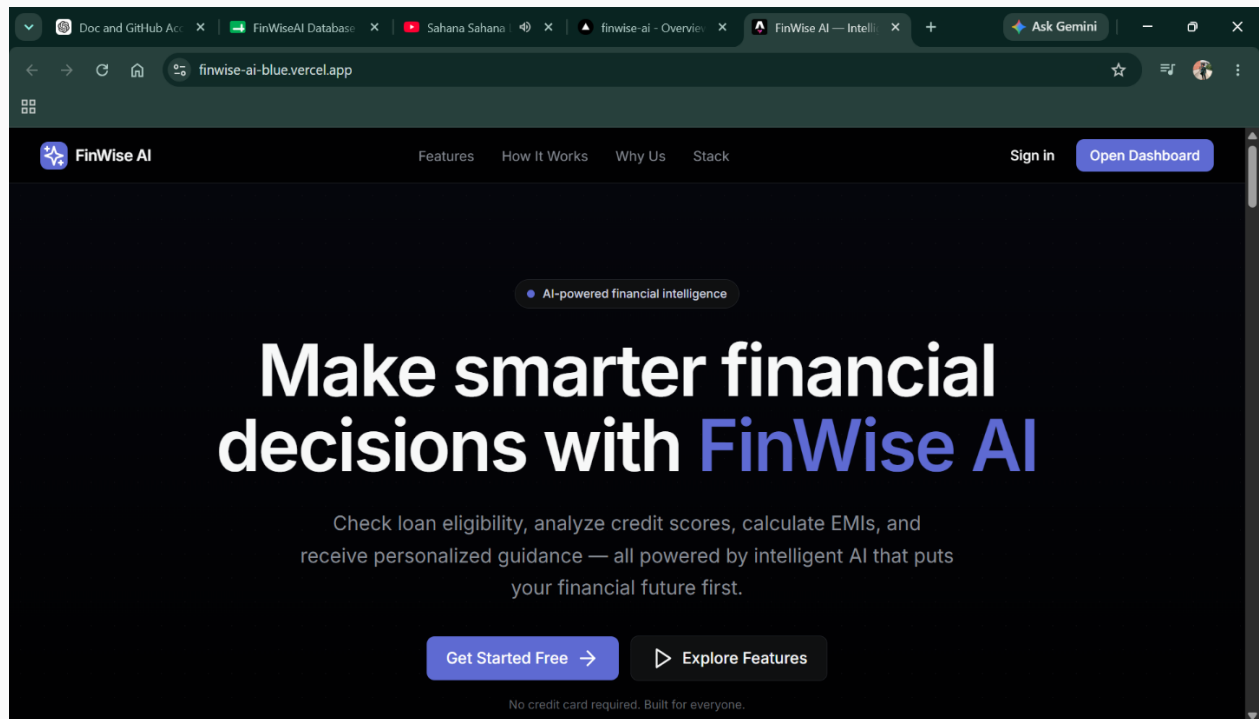


Figure 15.1 – Landing Page

15.2 Dashboard

The Financial Intelligence Dashboard is the hub a signed-in-feeling user lands on after the marketing page. It surfaces four KPI cards — Loan Eligibility (%), Credit Score, Estimated Monthly EMI, and AI Advisor status — a Recent Activity feed listing the last few completed analyses with their timestamps and sync status, and an AI Insights panel previewing personalised observations such as reducing credit utilisation or a home loan eligibility ceiling.

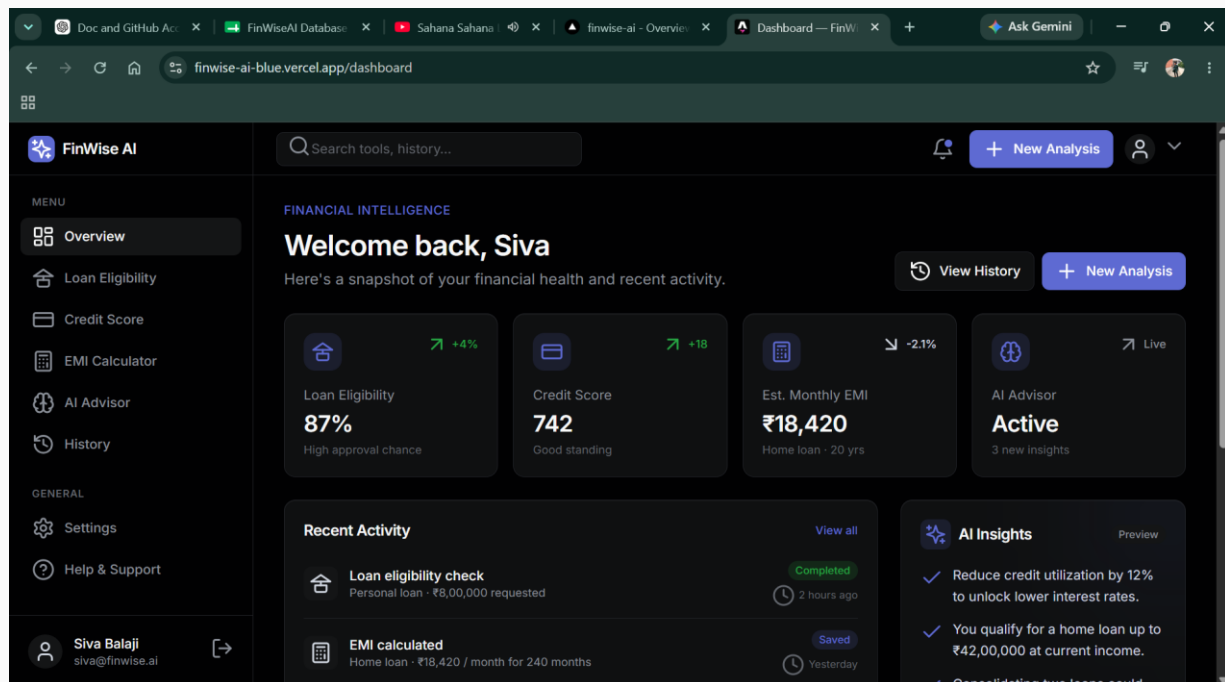


Figure 15.2 – Dashboard

15.3 Loan Eligibility Result

The Loan Eligibility page presents the applicant form (name, age, employment type, monthly income, existing EMI, credit score, loan amount, tenure, and purpose) alongside a live result panel showing the eligibility status, risk level, eligible amount, estimated EMI, interest rate, a bulleted “Reasoning” section explaining exactly why the result came out the way it did, and a “Next Suggested Action” callout.

The screenshot displays the 'Loan Eligibility' page of the FinWise AI application. The interface is divided into a left sidebar with navigation options (Overview, Loan Eligibility, Credit Score, EMI Calculator, AI Advisor, History, Settings, Help & Support) and a main content area. The main content area is titled 'Loan Application' and includes a search bar and a 'New Analysis' button. The application form contains the following fields and values:

Field	Value
Full Name	bala
Age	23
Employment Type	Salaried
Monthly Income	₹ 450000
Existing Monthly EMI	₹ 1
Credit Score	786
Loan Amount	₹ 500000
Loan Tenure (months)	12
Loan Purpose	

The results panel on the right shows the following information:

- ELIGIBILITY STATUS:** Eligible (Low Risk - DTI 10%)
- Eligible Amount:** ₹25,32,381
- Estimated EMI:** ₹44,424
- Risk Level:** Low Risk
- Interest Rate:** 12.0%

REASONING:

- ✓ Strong credit profile and healthy debt-to-income ratio.
- ✓ Requested amount is within your eligible limit of ₹25,32,381.

Next Suggested Action: You can proceed with your loan application.

Figure 15.3 – Loan Eligibility Result

15.4 Credit Score Analysis

The Credit Score page takes a raw score plus income, existing EMI, active loan count, credit card utilisation, and recent enquiries, and returns a large credit-rating gauge (for example “787 / 900 — Very Good”), an approval-probability figure, a financial-health progress bar, and side-by-side Positive Factors and Negative Factors lists with an Improvement Recommendations section beneath them.

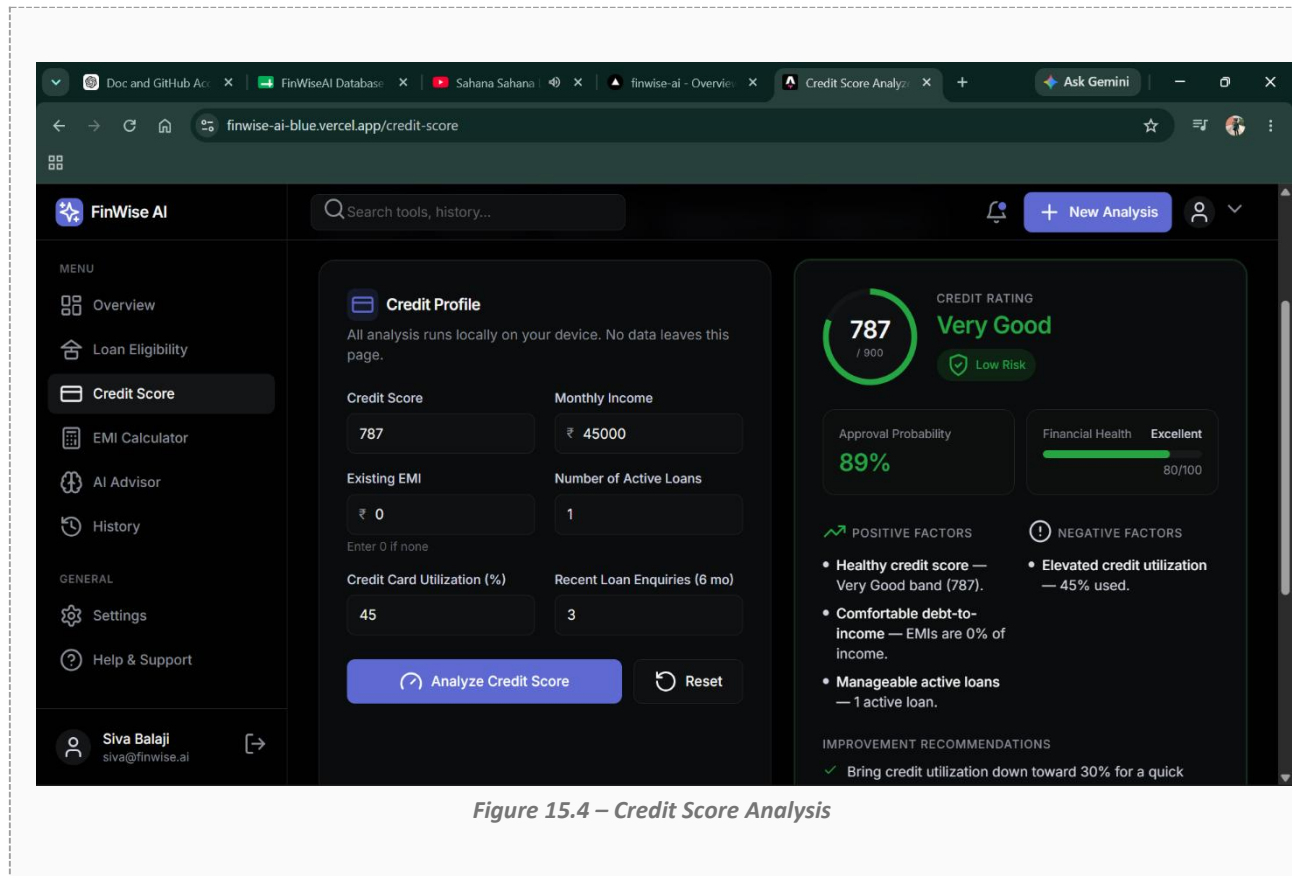


Figure 15.4 – Credit Score Analysis

15.5 EMI Calculator

The EMI Calculator accepts a loan amount, annual interest rate, tenure (in months or years), and loan type, and immediately returns the monthly EMI alongside a full Loan Summary — total interest, total payment, principal, interest percentage, loan type, and payment frequency — with a visual payment breakdown at the bottom of the result panel.

The screenshot displays the FinWise AI EMI Calculator interface. The browser address bar shows the URL `finwise-ai-blue.vercel.app/emi-calculator`. The application has a dark theme and a sidebar menu on the left with options: Overview, Loan Eligibility, Credit Score, EMI Calculator (selected), AI Advisor, History, Settings, and Help & Support. The user profile at the bottom left is Siva Balaji (siva@finwise.ai).

The main content area is titled "EMI Details" and includes a search bar "Search tools, history...". Below the title, it states "All calculations run locally on your device. No data leaves this page." The input fields are as follows:

- Loan Amount: ₹ 500000
- Annual Interest Rate (%): 12
- Loan Tenure: 24
- Tenure Unit: Months (dropdown)
- Loan Type: Home Loan (dropdown)

Buttons for "Calculate EMI" and "Reset" are located below the input fields. The results panel on the right shows a green success message "EMI calculated successfully" and the "MONTHLY EMI" of ₹23,537, with the note "Estimated monthly instalment for your loan." Below this, a table displays the following values:

Total Interest	Total Payment	Principal	Interest %
₹64,888	₹5,64,888	₹5,00,000	11.5%

The "LOAN SUMMARY" section includes a table with the following details:

Loan Type	Interest Rate	Duration	Payment Frequency
Home Loan	12.00% p.a.	24 months (24 mo)	Monthly

The "PAYMENT BREAKDOWN" section is partially visible at the bottom.

Figure 15.5 – EMI Calculator

15.6 AI Advisor

The AI Advisor page presents a chat-style interface with a set of suggested questions (“Can I improve my credit score?”, “How can I reduce my EMI?”, “Should I apply for this loan?”, “How much loan should I take?”) alongside a free-text input capped at 500 characters. Responses stream in progressively under the four fixed headings — Financial Advice, Risks, Suggestions, and Action Plan — and a toast confirms once the exchange has been saved to cloud history.

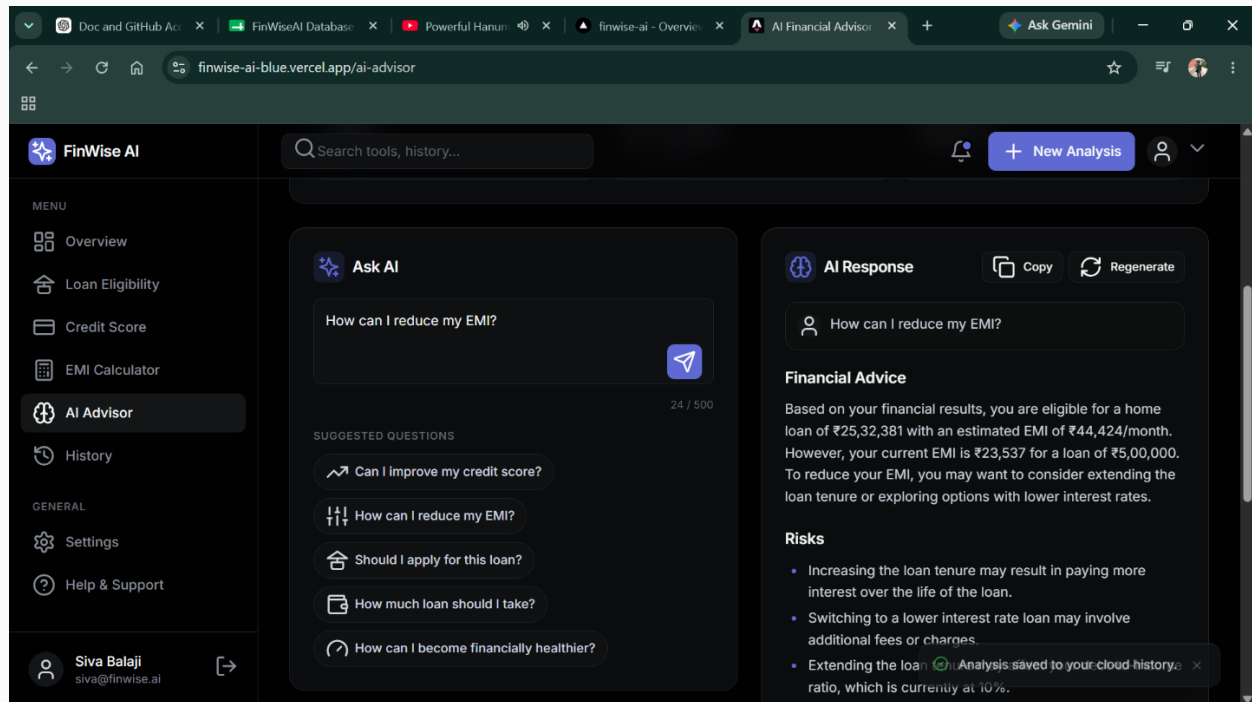
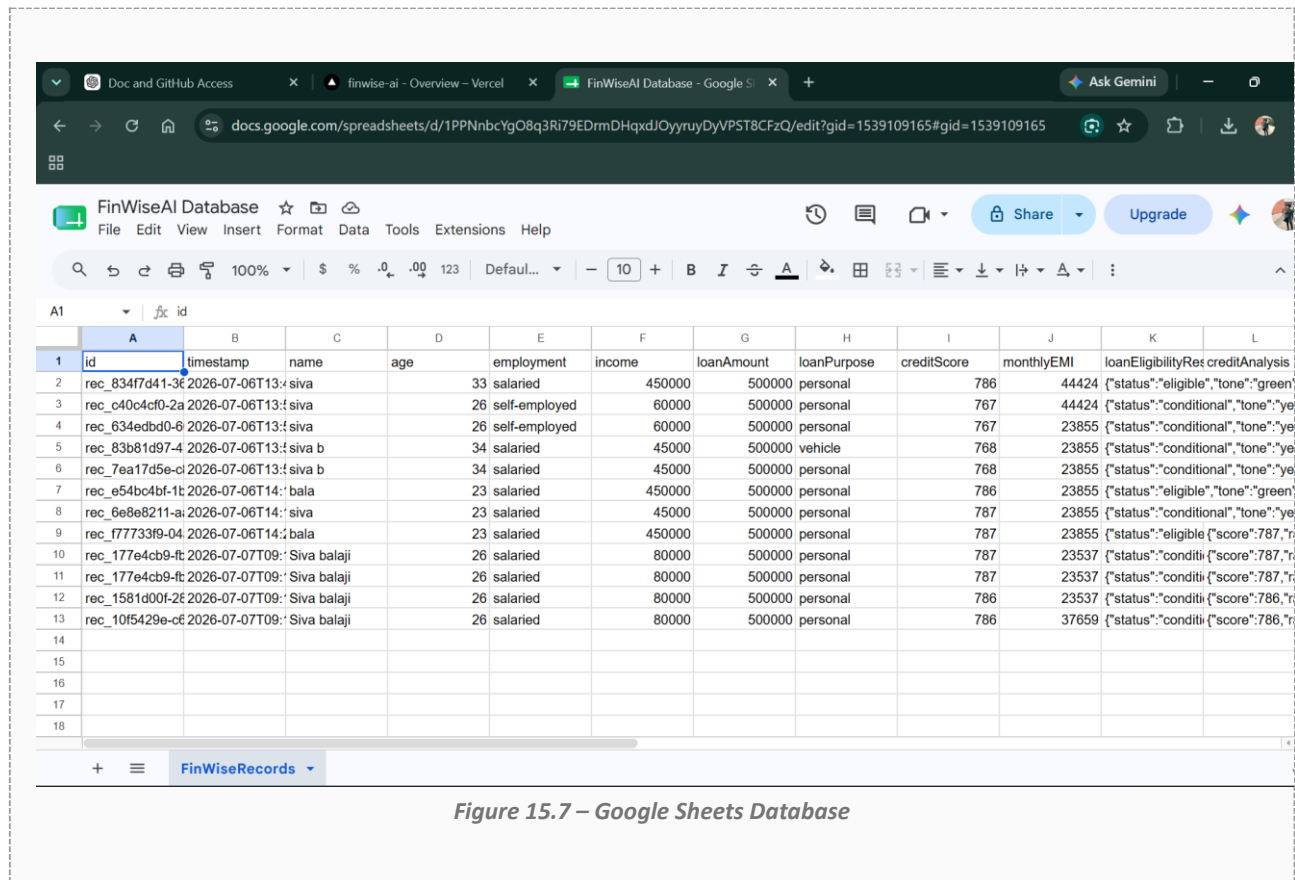


Figure 15.6 – AI Advisor

15.7 Google Sheets Database

Behind the History page sits a live Google Sheet (FinWiseRecords) that mirrors every synced analysis exactly as it was sent from the browser — id, timestamp, applicant name, age, employment, income, loan amount, purpose, credit score, monthly EMI, and the full JSON of the loan eligibility result and credit analysis for that row. This is the same sheet the Apps Script backend described in Chapter 11 reads from and writes to.



15.8 History Page

The History page lists every analysis the user has run, newest first, with a search bar (by name, status, rating, or summary), a filter dropdown (“All analyses”), and a per-record summary showing the module badges involved (Loan, Credit, EMI, AI), the loan eligibility verdict, and the credit rating, with a “View” action opening the full detail for that record. A persistent “All changes saved” indicator reflects the offline-first sync status described in Chapter 13.

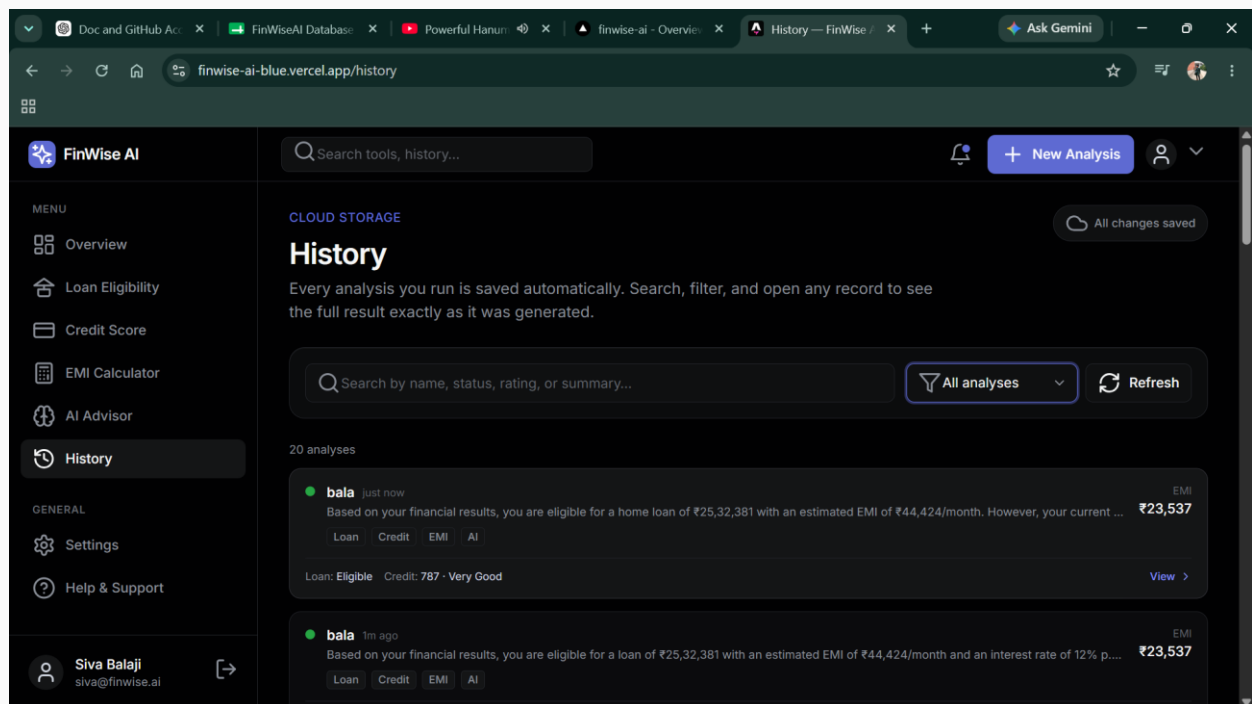


Figure 15.8 – History Page

15.9 Notifications

A notifications panel, accessible from the bell icon in the top bar on every dashboard page, is reserved for system messages — the current build shows a clean “No notifications yet — you're all caught up” state, with the panel wired to be populated as future features (for example proactive AI insights) are added.

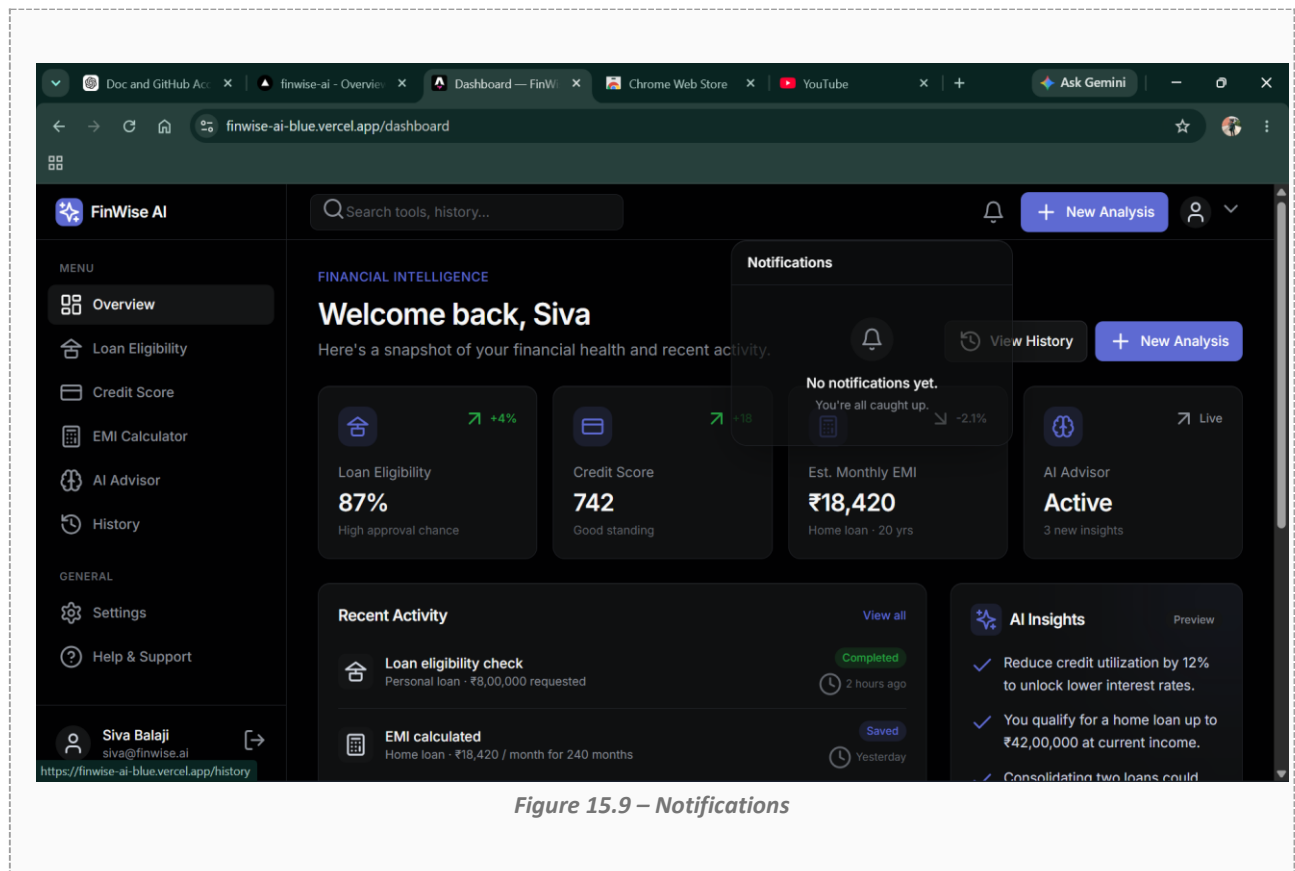


Figure 15.9 – Notifications

15.10 Mobile View

The same landing page, rendered on a mobile viewport, demonstrates the responsive behaviour described in Chapter 8.4: the hero section, call-to-action buttons, and dashboard preview card all reflow into a single-column, thumb-friendly layout without any separate mobile-only markup — the same Astro components and Tailwind breakpoints power both experiences.

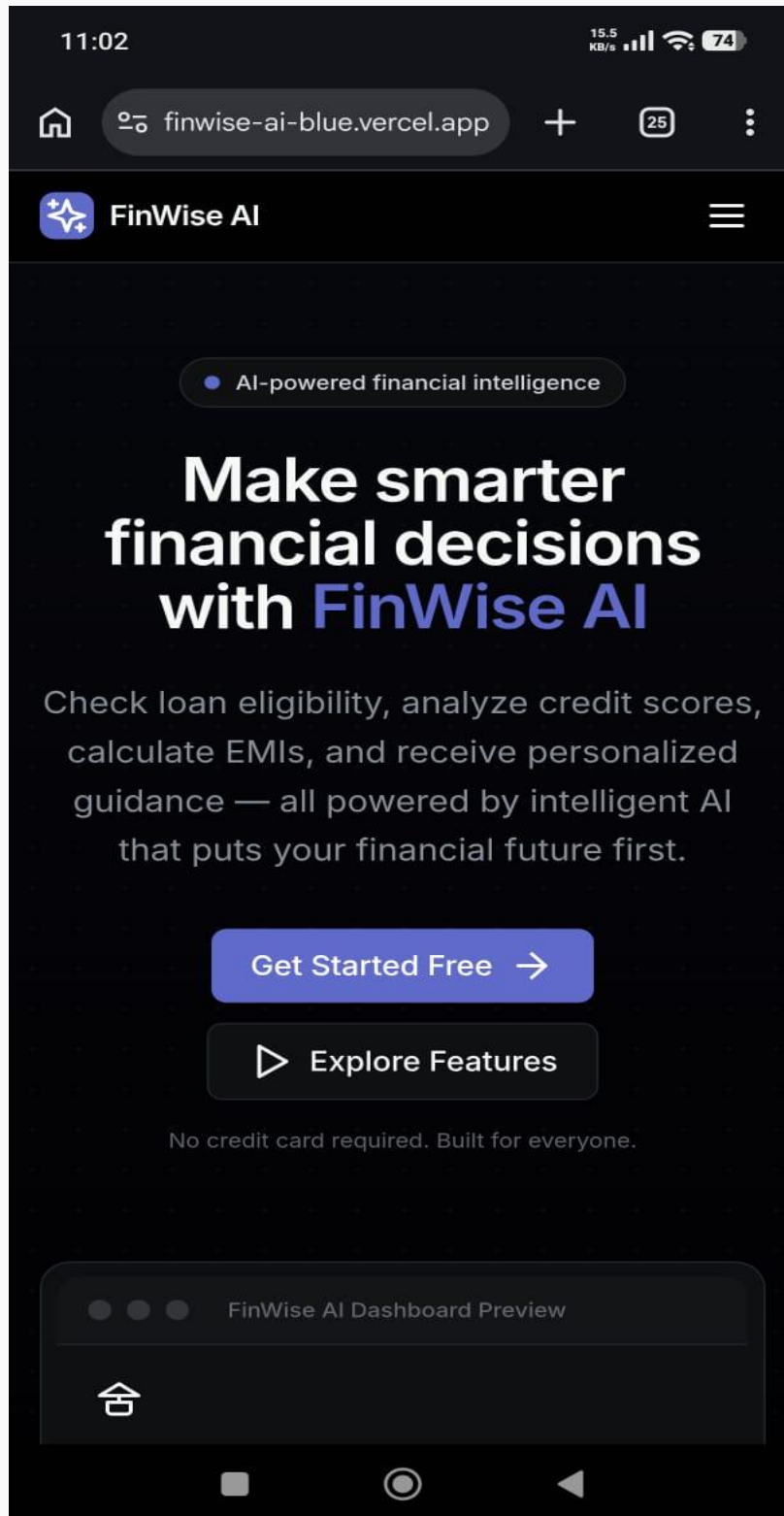


Figure 15.10 – Mobile View

16. Testing

Testing on this project was carried out manually and iteratively rather than through an automated test suite, since building one was explicitly scoped out for this internship cycle (see Chapter 4.2). That did not mean testing was informal — each module was verified against hand-calculated examples before being wired into the next layer of the application, and the full application was walked through repeatedly across devices, browsers, and network conditions before deployment. This chapter documents that process in the same structure it was actually carried out: functional correctness first, then the UI and responsive behaviour, then browser compatibility, then the two API routes, then the Google Sheets integration, and finally the AI prompt behaviour.

16.1 Functional Testing

Functional testing focused on the three deterministic engines, since a wrong number here is the highest-impact class of bug in the entire application. Each calculator was tested against manually worked examples spanning clear-approval, conditional, and rejection cases.

Module	Test Case	Input Summary	Expected Result	Outcome
Loan Eligibility	Clean approval	Age 23, income ₹4,50,000, score 786, EMI ₹1	Eligible, Low Risk, high eligible amount	Pass
Loan Eligibility	Conditional — amount above limit	Age 27, income ₹45,000, score 789, requested ₹5,00,000	Eligible with Conditions, amount reduced	Pass
Loan Eligibility	Rejected — below minimum age	Age 19, otherwise strong profile	Rejected, age policy reason shown	Pass
Loan Eligibility	Rejected — high risk credit score	Score 610, income ₹40,000	Rejected, High Risk reason shown	Pass
Credit Score	Very Good band	Score 787, income ₹45,000, utilisation 45%	Rating Very Good, Low Risk, 89% approval	Pass
Credit Score	Poor band, high utilisation	Score 590, utilisation 85%, 5 enquiries	Rating Poor, several negative factors listed	Pass
EMI Calculator	Home loan, 24-month tenure	₹5,00,000 at 12% for 24 months	EMI ₹23,537/month, matches hand calculation	Pass
EMI Calculator	Zero-interest edge case	Rate = 0%	EMI = principal / tenure exactly	Pass
EMI Calculator	Year-to-month tenure conversion	20 years selected	Converted correctly to 240 months	Pass

Every EMI figure produced by the Loan Eligibility Checker was additionally cross-checked against the standalone EMI Calculator using the same principal, rate, and tenure, confirming the shared `calculateEmi()` function returns identical results from both entry points — this was the single most important functional check given the two modules are used together so frequently.

16.2 UI Testing

UI testing verified that every interactive element behaves correctly across its states — default, hover, focus, loading, and result-displayed — and that the glassmorphism design system (Chapter 8) renders consistently across pages.

Area	What Was Verified	Result
Form validation	Empty required fields, out-of-range numbers, and invalid combinations show inline errors without a page reload	Pass
Loading states	Submit buttons disable and show a spinner while a request is in flight, and re-enable in the finally block regardless of success or failure	Pass
Result panels	Status tone (green/yellow/red) matches the returned eligibility/credit tone exactly	Pass
Toast notifications	“Analysis saved to your cloud history” toast appears after every successful save and dismisses automatically	Pass
AI chat rendering	Markdown headings, bullet lists, and numbered lists in the AI response render with correct formatting as they stream in	Pass
Sidebar navigation	Active route is highlighted correctly across Overview, Loan Eligibility, Credit Score, EMI Calculator, AI Advisor, and History	Pass

16.3 Responsive Testing

Every page was checked at three representative breakpoints — a typical desktop width, a tablet width, and a common mobile width — to confirm the CSS Grid and Flexbox layouts described in Chapter 8.4 reflow correctly rather than simply shrinking.

Breakpoint	Representative Width	Verified Behaviour
Desktop	1440px and above	Two-column dashboard with sticky sidebar; result panels shown side-by-side with input forms
Tablet	768px – 1024px	Sidebar collapses to icon-only or top navigation; KPI cards reflow to a two-column grid
Mobile	360px – 428px	Single-column layout throughout; forms and result cards stack vertically; touch targets remain large enough for comfortable tapping

The landing page and dashboard were both confirmed on an actual Android device (Poco F6) in addition to browser dev-tools emulation, which surfaced small spacing issues around the hero call-to-action buttons that were not visible in the emulated viewport — these were corrected before the final deployment.

16.4 Browser Compatibility Testing

Browser	Platform	Result	Notes
Google Chrome	Windows / Android	Pass	Primary development and testing browser

Browser	Platform	Result	Notes
Microsoft Edge	Windows	Pass	Chromium-based; identical rendering to Chrome as expected
Mobile Chrome	Android	Pass	Verified on-device; SSE streaming for the AI advisor confirmed working over mobile data and Wi-Fi

Because the application avoids exotic browser APIs — relying on `fetch`, `ReadableStream`, `localStorage`, and standard CSS Grid/Flexbox — no browser-specific workarounds were required. The one area given particular attention was Server-Sent-Event parsing in `groq.ts`, since streaming response handling has historically been a source of cross-browser inconsistency; no issues were observed in the browsers tested above.

16.5 API Testing

Both server routes were tested directly, independent of the UI, to confirm their contracts hold under both correct and malformed input.

Endpoint	Test	Expected Response	Result
<code>/api/advice</code>	Valid question + context	200, streamed Markdown text under the four fixed headings	Pass
<code>/api/advice</code>	Empty question	400, “Your question cannot be empty.”	Pass
<code>/api/advice</code>	Question over 500 characters	400, length-limit error message	Pass
<code>/api/advice</code>	GROQ_API_KEY temporarily unset	503, generic “temporarily unavailable” message, no key leaked	Pass
<code>/api/history</code>	POST valid record	200, { ok: true }	Pass
<code>/api/history</code>	POST malformed record (no id)	400, { ok: false, error }	Pass
<code>/api/history</code>	GET with no backend configured	200, { ok: true, configured: false, records: [] }	Pass
<code>/api/history</code>	GET with backend configured	200, full record list, newest first	Pass

16.6 Google Sheets Testing

The cloud persistence path was tested end-to-end, from a browser action through to the actual sheet contents, in both the online and offline case.

Scenario	Steps	Result
Normal save	Complete a loan eligibility check while online	Record appears in <code>localStorage</code> immediately, then in the <code>FinWiseRecords</code> sheet within seconds, marked synced
Offline save	Disable network, complete a credit analysis	Record saved locally as pending; toast confirms local save only
Reconnect sync	Re-enable network after an offline save	The online event listener fires <code>syncPending()</code> , and the queued record appears in the sheet without user action

Scenario	Steps	Result
Duplicate save	Re-run the same analysis twice	Sheet row is updated in place (upsert by id) rather than duplicated
Sanitisation	Attempt to save a record with an oversized advice string and control characters	String is trimmed to the 8,000-character cap and control characters are stripped before reaching the sheet
Unconfigured backend	Remove GOOGLE_SCRIPT_URL and reload History	History still renders from the local mirror with no error state shown

16.7 AI Prompt Testing

Prompt testing focused less on “does the model answer” and more on whether the model respects the constraints the system prompt places on it — the fixed heading structure, the instruction not to recalculate figures, and the length limit.

Test	What Was Checked	Result
Heading contract	Response uses exactly the four headings (Financial Advice, Risks, Suggestions, Action Plan) in order	Pass across repeated runs
Numeric fidelity	Figures mentioned in the AI response (EMI, eligible amount, credit score) match the values passed in FinancialContext exactly	Pass — no instance of the model inventing or altering a figure
No-context fallback	Question asked with no prior calculator results available	Model gives general guidance and directs the user to the calculators, as instructed
Length discipline	Response length stays close to the ~350-word target set in the system prompt	Pass, with minor natural variance
Currency formatting	Amounts are expressed using ₹ as instructed	Pass
Tone	Advice is encouraging but does not promise guaranteed outcomes, as required by the system prompt	Pass

16.8 Summary of Testing

Across functional, UI, responsive, cross-browser, API, cloud-sync, and AI-prompt testing, no defect was found that required a change to the core financial formulas or the security boundaries described in Chapters 6 and 12 — the issues that did surface during this phase were primarily around deployment configuration, and are documented in detail in Chapter 18 (Challenges Faced), since they were ultimately more instructive than the testing itself.

17. Deployment

17.1 Introduction

FinWise AI is deployed on Vercel, chosen specifically because the @astrojs/vercel adapter allows Astro's hybrid rendering model — static pages plus two on-demand functions — to deploy from a single repository without any separate infrastructure for the API routes.

17.2 Deployment Steps

7. Push the repository to GitHub, with the Astro project living inside an `app/` subdirectory rather than the repository root.
8. Import the repository into Vercel and set the project's root directory explicitly to `app/`, since Vercel otherwise assumes the Astro project sits at the repository root.
9. Add the required environment variables in the Vercel dashboard: `GROQ_API_KEY` (required), and `GOOGLE_SCRIPT_URL` / `GOOGLE_SCRIPT_TOKEN` (optional — enable cloud history).
10. Trigger the deployment. Vercel runs `astro build`, producing pre-rendered HTML for every static route and packaging `/api/advice` and `/api/history` as individual serverless functions.
11. Verify the live deployment: confirm the landing page, dashboard, and all three calculators render correctly, and that a test question to the AI advisor streams a response.

17.3 Environment Configuration

Variable	Required	Description
<code>GROQ_API_KEY</code>	Yes	Server-only Groq API key for the AI advisor; never prefixed with <code>PUBLIC_</code>
<code>GOOGLE_SCRIPT_URL</code>	No	Deployed Apps Script Web App <code>/exec</code> URL; if unset, history is local-only
<code>GOOGLE_SCRIPT_TOKEN</code>	No	Optional shared secret matched against <code>SCRIPT_TOKEN</code> in Apps Script

17.4 Google Sheets Setup

Separately from the Vercel deployment, a Google Sheet was created with columns matching the `AnalysisRecord` schema, and the Apps Script in `docs/google-apps-script.gs` was deployed as a Web App (Execute as: Me, Who has access: Anyone) so the `/exec` URL could be pasted into `GOOGLE_SCRIPT_URL` without requiring end users to authenticate with Google themselves.

17.5 Post-Deployment Verification

After the first successful deployment, the live site (`finwise-ai-blue.vercel.app`) was walked through using the same checklist described in Chapter 16 — functional correctness of all three calculators, a live AI advisor conversation, and a Google Sheets write — to confirm the production build behaved identically to the local development build.

18. Challenges Faced

No non-trivial deployment goes smoothly on the first attempt, and FinWise AI was no exception. This chapter documents the real issues encountered while building and shipping the project — not as a formality, but because working through them was where most of the actual learning in this internship happened.

18.1 Astro + Vercel Root Directory Misconfiguration

The very first deployment attempt failed at the build step. The repository is structured with the Astro project living inside an `app/` subdirectory (to keep `.ai/` planning documents, `docs/`, and the top-level `README` separate from the application source), but Vercel's default import assumes the framework's project root is the repository root. Vercel could not locate `package.json` or `astro.config.mjs`, and the build failed before a single line of application code ran.

18.2 Astro Build Configuration Errors

Once the root directory issue was identified, a second round of build failures appeared — this time from the `@astrojs/vercel` adapter itself, which needs to know which routes are meant to be pre-rendered and which are meant to run on demand. Early in development, the two API routes did not yet have `export const prerender = false` set, so Astro attempted to statically render `/api/advice` and `/api/history` at build time, which failed immediately since those routes depend on a request object and a runtime environment that does not exist during a build.

18.3 Google Apps Script Authorization Issues

The first deployed version of the Apps Script Web App was configured with the default “Execute as: User accessing the web app” permission. Because the application is meant to be used by anonymous visitors who have never signed in with a Google account, every request to the sheet failed with an authorization prompt instead of returning data — the Apps Script was, in effect, asking an anonymous browser request to log in to Google, which it obviously could not do.

18.4 History Synchronisation Bug — Duplicate Records

During early testing of the offline-first flow, re-running the same loan eligibility check twice in a row (for example, adjusting one field and resubmitting) produced two separate rows in the Google Sheet instead of one updated row. Investigating the Apps Script code showed that the very first version of the backend used a simple `sheet.appendRow()` for every save request, with no lookup against existing rows by id — every save was treated as a brand-new record regardless of whether that id already existed in the sheet.

18.5 “Delete Sync” Implementation Gap

A related issue surfaced once manual row deletion was tested directly in the Google Sheet (to simulate an admin cleaning up test data): a record deleted from the sheet would silently reappear in the app's History page on the next load. The local mirror in `localStorage` had marked that record synced at some point in the past, and the original reconciliation logic only ever added cloud records into the local mirror — it never removed a locally-cached synced record that had disappeared from the cloud.

18.6 Environment Variable Exposure Risk

During an early refactor, `GROQ_API_KEY` was briefly read using `import.meta.env.PUBLIC_GROQ_API_KEY` in one experimental branch of the AI service while testing whether the advisor could be called directly from client-side code for lower latency. This was caught during a self-review before it was ever deployed, but it highlighted how easy it is, in an Astro project, to accidentally cross the `PUBLIC_` boundary and ship a secret to every visitor's browser.

18.7 API Integration Debugging — Streaming Response Truncation

In the earliest working version of the AI advisor, long responses would occasionally cut off mid-sentence in the browser, even though server logs showed Groq had sent the full response. The SSE parser in `groq.ts` was initially splitting incoming data strictly on `'\n'`, rather than on the blank-line boundary that actually separates SSE frames, which meant a data: frame that happened to be split across two TCP chunks by the network layer could be parsed incompletely and silently dropped.

18.8 Credit and Loan Modules Disagreeing on EMI

During an early integration pass, a discrepancy was noticed between the EMI shown on the Loan Eligibility result screen and the EMI shown by the standalone EMI Calculator for what should have been the same loan amount, rate, and tenure. The root cause was that the eligibility module had originally implemented its own, slightly different rounding approach to the EMI formula rather than importing `calculateEmi()` from a single shared location — a small divergence that nonetheless produced a one-rupee difference in some cases, which was enough to look like a bug to a careful reviewer.

19. Solutions Implemented

Each of the issues described in Chapter 18 was traced to a specific, fixable cause rather than patched around superficially. This chapter documents the actual fix applied for every challenge, in the same order.

19.1 Fix — Vercel Root Directory

The project's root directory was explicitly set to `app/` in the Vercel project settings (Settings → General → Root Directory), rather than relying on auto-detection. This single configuration change let Vercel correctly locate `package.json`, `astro.config.mjs`, and the `src/` tree, and the build proceeded normally from that point on.

19.2 Fix — Prerender Flags on API Routes

`export const prerender = false` was added explicitly to both `src/pages/api/advice.ts` and `src/pages/api/history.ts`, which tells Astro to treat these two files as on-demand server endpoints rather than attempting to pre-render them as static HTML. Every other route in the application intentionally keeps the default (`prerender = true`), which is what allows the marketing pages and calculators to ship as fast, static HTML while only these two routes run per-request on the server.

19.3 Fix — Apps Script Execution Identity

The Apps Script Web App deployment was changed to Execute as: Me and Who has access: Anyone, which means every request runs under the developer's own Google identity regardless of who is calling it, and does not require the calling browser to be signed in at all. This matches the intended trust model exactly: the application, not the end user, owns the underlying Google Sheet.

19.4 Fix — Upsert-by-ID in Apps Script

The Apps Script save handler was rewritten to search the id column for an existing match before writing: if a matching row is found, its cells are overwritten in place; only if no match exists is a new row appended. This single change eliminated duplicate records entirely and made repeated saves of the same analysis (for example, after asking the AI advisor a follow-up question that updates the aiAdviceSummary field) behave as an update rather than an insert.

19.5 Fix — Cloud-Authoritative Reconciliation

The reconcileWithCloud() logic in history.ts was rewritten so that every cloud record is written back into the local mirror as synced, and any previously-synced local record that is no longer present in the cloud list is dropped rather than kept. Only records that have never successfully synced (pending or error) survive a reconciliation pass if they are absent from the cloud — this is exactly the “genuine offline record still waiting to upload” case the design needs to protect, while correctly discarding records that were deleted server-side.

19.6 Fix — Enforcing the PUBLIC_ Boundary

The experimental client-side branch reading a PUBLIC_-prefixed Groq key was removed entirely, and the project rules were updated (documented in .ai/project_rules.md) to state explicitly that GROQ_API_KEY and GOOGLE_SCRIPT_URL must only ever be read inside groq.ts and googleSheets.ts respectively, both of which are exclusively imported by server-only modules. This is also why the AI advisor is architected the way it is described in Chapter 10 — a single provider file is the only place the key can leak from, which makes it easy to audit.

19.7 Fix — Correct SSE Frame Parsing

parseSseStream() in groq.ts was rewritten to buffer incoming bytes and split on the double-newline ('\n\n') sequence that actually delimits SSE frames per the specification, rather than splitting on every single newline. Partial frames left over after a split are retained in the buffer and prefixed onto the next chunk of data, so a frame arbitrarily split across two network reads is always reassembled correctly before being parsed as JSON.

19.8 Fix — Single Shared EMI Function

The eligibility module's locally-implemented EMI calculation was deleted, and utils/loan.ts was updated so that calculateEmi() is the one and only implementation of the formula in the codebase; utils/emi.ts now explicitly imports and re-exports it rather than reimplementing it. This is the change reflected in the current codebase and described in Chapters 9.1 and 9.3 — it is also why the report can state with confidence that the two modules can never disagree on an EMI figure going forward.

19.9 Outcome

Every issue in this chapter was resolved before the final deployment described in Chapter 17, and each fix is reflected in the current, live version of the codebase rather than being a workaround layered on top of it. Taken together, they were the most valuable part of the internship for me personally — the calculators and the AI integration were the parts I had planned for from the start, but the deployment and synchronisation issues were the parts that taught me the most about building something that has to actually run correctly for someone else.

20. Learning Outcomes

Beyond the finished product, this internship was the first time I built a multi-module application where the correctness of the core logic genuinely mattered — a wrong number here is not a cosmetic bug — while also integrating a generative AI layer on top of it responsibly. The specific, concrete skills this project reinforced were:

20.1 Astro and Rendering Models

I gained hands-on experience with Astro's static-site-generation and server-side-rendering hybrid model: understanding exactly which pages benefit from zero-JS pre-rendering, which routes must opt out with `prerender = false`, and why that boundary is also a security boundary and not just a performance one.

20.2 TypeScript in a Financial Context

Writing strictly-typed policy constants, discriminated unions for statuses and risk levels, and pure functions with fully-typed return shapes gave me a much better appreciation for why type safety matters more, not less, in domains where a wrong value has real consequences.

20.3 Tailwind CSS v4 and Design Systems

Building the entire visual identity through a single `@theme` token file, rather than styling each page independently, taught me how a small design system pays for itself very quickly once more than a handful of pages need to share a consistent look.

20.4 Responsible AI Integration

Perhaps the most important lesson of the project was architectural: generative AI should narrate results, not compute them. Designing the AI advisor so it is structurally incapable of overriding a number the deterministic engines already produced — by construction, not just by prompt instruction — was a genuinely new way of thinking about where an LLM belongs in a product.

20.5 Provider Abstraction and Streaming

Implementing the `provider.ts` single-switch-point pattern, and building a correct Server-Sent-Events parser from first principles (including fixing the frame-splitting bug described in Chapter 18.7), gave me a much deeper understanding of how streaming APIs actually work under the hood, beyond just calling a client library.

20.6 Serverless and Offline-First Architecture

Building the Google Sheets integration end-to-end — from the Apps Script backend, through server-side sanitisation, to an offline-first client that queues, retries, and reconciles against a cloud source of truth — taught me patterns (upsert-by-id, sync state machines, cloud-authoritative reconciliation) that apply well beyond this one project.

20.7 Deployment and Production Debugging

Resolving the Vercel root-directory issue, the Apps Script authorization problem, and the duplicate-record bug under real deployment conditions taught me more about how these pieces actually fit together than building the same features would have taught me if everything had simply worked the first time.

21. Future Scope

The items below are drawn directly from the “Future Improvements” section already documented in the project's own README, expanded with the specific architectural reasoning for why the current codebase is positioned to support each of them.

21.1 User Authentication

Adding user accounts would let history be partitioned per user rather than shared across one device-local mirror and one shared sheet. Because `services/history.ts` already treats “source of truth” as an abstraction (currently the shared Google Sheet), introducing per-user scoping would primarily mean adding a user identifier to the `AnalysisRecord` schema and to the Apps Script's row-matching logic, rather than a rewrite of the sync engine itself.

21.2 Interactive Charts

An amortisation schedule (principal vs. interest over time) and a credit-score trend chart are natural extensions of data the application already computes — `calculateEmiBreakdown()` already returns `totalInterest` and `principalPercentage`, and each analysis is already timestamped in history, so a trend chart is primarily a visualisation task on top of existing data rather than new backend work.

21.3 AI-Powered EMI Optimisation

A natural next step for the AI advisor is a dedicated “what if” mode — pre-payment suggestions, or balance-transfer analysis between two EMI scenarios — which the existing provider abstraction and prompt-building pattern in `advisor-prompt.ts` could support by adding a new fact-block formatter, without touching the Groq integration itself.

21.4 Light Mode

Because every colour in the interface is already expressed as a Tailwind `@theme` token rather than a hard-coded value, a light theme is primarily a matter of defining a second token set and a toggle, rather than restyling individual components.

21.5 Mobile PWA with Offline Calculators

The three deterministic calculators already run entirely client-side with no network dependency, which means they are already, functionally, offline-capable — the remaining work for a true PWA is a service worker and an app manifest, not a rewrite of the calculation logic.

21.6 Real Banking API Integration

Replacing the user-entered credit score with a live pull from a credit bureau, or connecting to real account data through frameworks like Account Aggregator, would sit cleanly behind the same `FinancialContext` abstraction the AI advisor already consumes — the rest of the application would not need to know whether a score was typed in by a user or fetched from a bureau.

21.7 Automated Testing

Given how central the pure functions in `utils/loan.ts`, `credit.ts`, and `emi.ts` are to the whole application, they are also the easiest and highest-value place to introduce automated unit tests, followed by Playwright-based end-to-end tests covering the manual test cases already documented in Chapter 16.

22. Conclusion

FinWise AI set out to answer a specific, practical question — where does a person actually stand, financially, before they walk into a bank — and to answer it with numbers a user can trust and an AI advisor that can explain those numbers rather than replace them. Over the course of this internship, that goal shaped almost every architectural decision described in this report: deterministic engines kept strictly separate from the generative AI layer, secrets kept server-side behind exactly two API routes, and a persistence layer designed to never lose a result even when the network does not cooperate.

The final application — live at finwise-ai-blue.vercel.app — implements three fully working financial calculators, a streaming AI Financial Advisor grounded in the user's own results, an offline-first history synchronised to a Google Sheet, and a dashboard tying all of it together, all wrapped in a consistent, dark glassmorphism interface that holds up across desktop, tablet, and mobile. Just as importantly, the issues documented in Chapters 18 and 19 — the Vercel root-directory misconfiguration, the Apps Script authorization problem, the duplicate-record bug, the SSE parsing fix — were not glossed over, because they were the parts of the internship that most closely resembled the debugging work of an actual production deployment rather than a classroom exercise.

I came into this internship able to write working code; I am finishing it with a much clearer sense of how to build something that is also correct, secure, and resilient when things go wrong outside my control. FinWise AI is not a finished financial product, but it is, deliberately, an honest attempt at one — and the architecture it is built on is one I would be comfortable extending, rather than rewriting, for every item listed in Chapter 21.

23. References

- Astro Documentation — <https://docs.astro.build>
- Astro Vercel Adapter — <https://docs.astro.build/en/guides/integrations-guide/vercel/>
- TypeScript Documentation — <https://www.typescriptlang.org/docs/>
- Tailwind CSS v4 Documentation — <https://tailwindcss.com/docs>
- Groq API Documentation — <https://console.groq.com/docs>
- Groq Chat Completions (OpenAI-compatible) Reference — <https://console.groq.com/docs/api-reference>
- Google Apps Script Documentation — <https://developers.google.com/apps-script>
- Google Sheets API / Apps Script Web Apps Guide — <https://developers.google.com/apps-script/guides/web>
- Vercel Documentation — <https://vercel.com/docs>
- MDN Web Docs — Server-Sent Events — https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- MDN Web Docs — Web Storage API (localStorage) — https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- Lucide Icons — <https://lucide.dev>

- FinWise AI Repository README.md and internal documentation (project source)
- SmartBridge × SkillWallet AI Vibe Coding Internship — project guidelines and templates